LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 5. Conditional Statements

Lecture: 46. String Comparison

Section 1: Lecture Summary

This lecture focuses on **string comparison** in Python, revisiting and expanding the use of relational (comparison) operators beyond integers to strings and other data types. It starts by reviewing the basic relational operators (`<`, `<=`, `>`, `>=`, `==`, `!=`) which return Boolean values and are used to compare values. The lecture then explains **how strings are compared lexicographically**, i.e., in dictionary order, based on the order in which words appear in a dictionary. Various examples illustrate that comparison considers character-by-character order, with numbers coming before uppercase letters, and uppercase letters coming before lowercase letters. The lecture also covers how relational operators work on other data types including integers, floats, booleans, and complex numbers — with the caveat that for complex numbers, only equality (`==`) and inequality (`!=`) comparisons are valid, since less than or greater than comparisons are undefined. The lecture provides practical examples, verifies results with Python code in Jupyter Notebook, and stresses the ability to compare mixed numeric types but not mix strings with numeric or boolean types for comparison.

Section 2: Key Concepts and Explanations

- **Relational Operators Overview**

  - `<` (less than), `<=` (less than or equal), `>` (greater than), `>=` (greater than or equal), `==` (equal), `!=` (not equal)

  - Return `True` or `False` based on comparison.

  - Commonly used with integers, but applicable to many other types.

- **String Comparison (Lexicographic/Dictonary Order)**

- Strings are compared lexicographically, meaning character-by-character like words in a dictionary.

- The string that would appear earlier in a dictionary is considered “less than” the other.

- Comparison proceeds character-by-character from the start until a difference is found.

- If all characters are the same but one string is shorter, the shorter string is less than the longer string.

- **Order Hierarchy in String Comparison**

- Numbers (`'0'` to `'9'`) come before uppercase letters (`'A'` to `'Z'`).

- Uppercase letters come before lowercase letters (`'a'` to `'z'`).

- **Examples in String Comparison**

- `'apple' < 'ball' < 'cat' < 'dog' < 'python'`

- `'apply' > 'Apple'` because `'y'` comes after `'e'`.

- `'cat' < 'catch'` since `'catch'` is longer but starts with `'cat'`.

- `'data' > 'Data'` because lowercase letters come after uppercase.

- `'2nd' < 'Byte'` numbers come before letters.

- **Relational Operators on Other Data Types**

- **Integer**: Fully supported.

- **Float**: Fully supported, similar to integer comparison.

- **Boolean**:

  - Internally, `True` = 1 and `False` = 0.

  - Comparisons like `False < True` yield `True`.

- **Complex Numbers**:

  - Only `==` and `!=` operators work.

  - `<`, `>`, `<=`, `>=` are not supported because order comparison is ambiguous.

- **Mixed Type Comparison**

  - Integer and float can be compared directly.

  - Integer and boolean can be compared (boolean acts like 0 or 1).

  - Strings can only be compared with strings.

Section 3: Example Code and Use Cases

String comparison examples

```
s1 = 'software'
s2 = 'Hardware'
print(s1 > s2)  # True because 's' > 'H'

s1 = 'python'
s2 = 'pycharm'
print(s1 < s2)  # False because 't' > 'c' at the differing character

s1 = 'integer'
s2 = 'integer'
print(s1 == s2)  # True because both strings are identical

s1 = 'printer'
s2 = 'print'
print(s1 > s2)  # True because 'printer' is longer but starts with 'print'
```

These examples show string comparison based on lexicographical order.

Boolean comparisons

```
print(False < True)   # True, because False is 0 and True is 1
print(True == True)   # True
print(False == False) # True
```

These demonstrate how relational operators apply to Boolean values.

Complex number equality and inequality

The following will raise TypeError if uncommented:

print(c1 < c2)

print(c1 > c2)

```
c1 = 3 + 4j
c2 = 4 + 3j
print(c1 == c2)  # False
print(c1 != c2)  # True
```

Complex numbers do not support ordered comparisons.

Mixed type comparisons

print('5' < 5) Error: cannot compare string and integer

```
print(5 < 5.5)      # True, integer and float comparison
print(True < 2)     # True, boolean treated as integer 1
```

You can compare numeric types and booleans, but not strings with numbers.

Section 4: Key Takeaways

- Relational operators return Boolean results when comparing values.

- String comparison uses lexicographic (dictionary) order, character by character.

- The order sequence for characters is: numbers < uppercase letters < lowercase letters.

- Strings are compared only against other strings.

- Boolean values behave like integers (`False=0`, `True=1`) in comparisons.

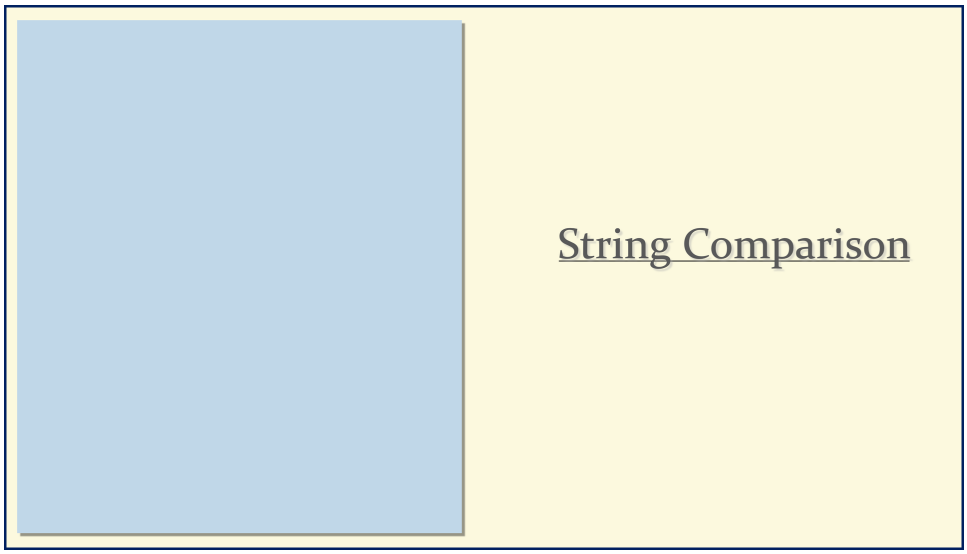
- Only `==` and `!=` operators work with complex numbers; relational inequalities are not supported.

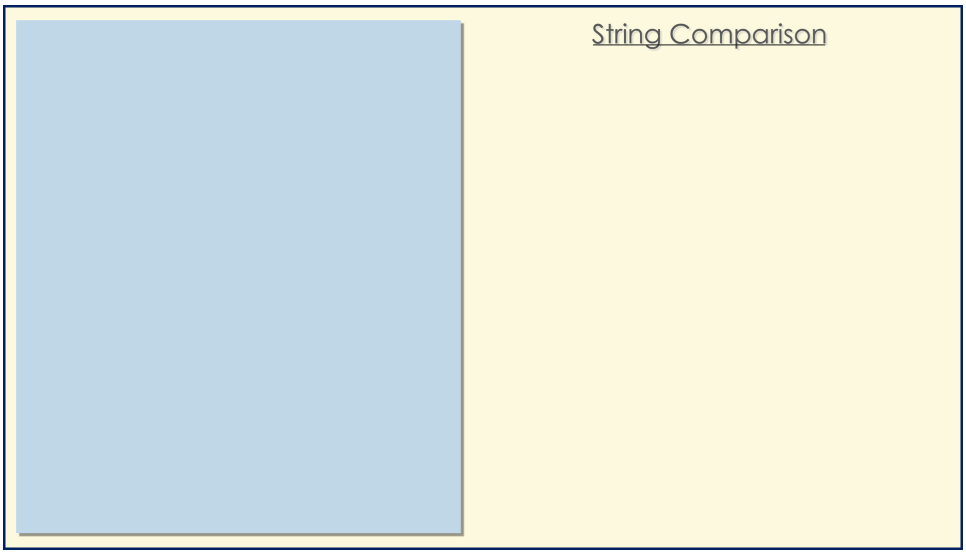
- Numeric comparisons allow mixing integers, floats, and booleans.

- Practice confirming these behaviors through code helps solidify understanding.

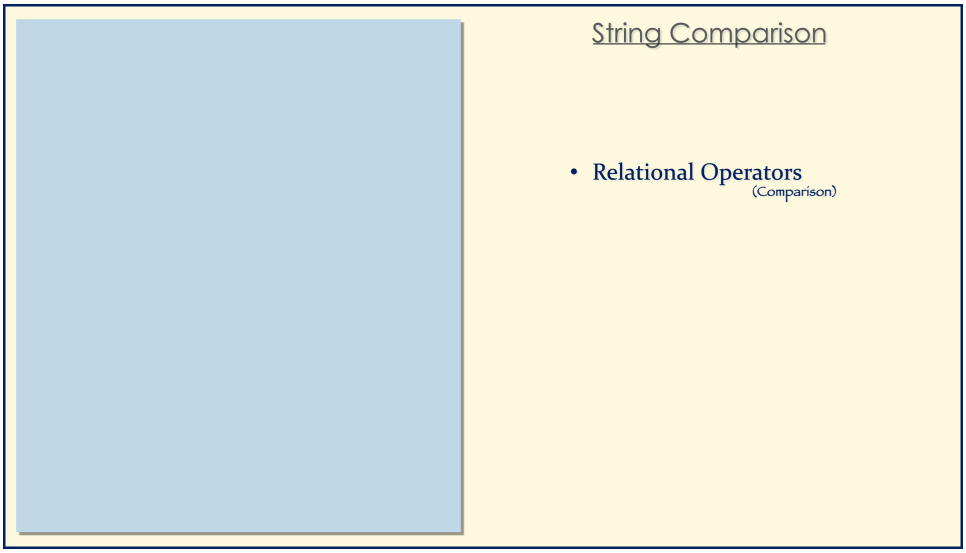
Lecture in Slides

Please Note: This document presents a curated selection of slides from the lecture; others have been excluded for conciseness.



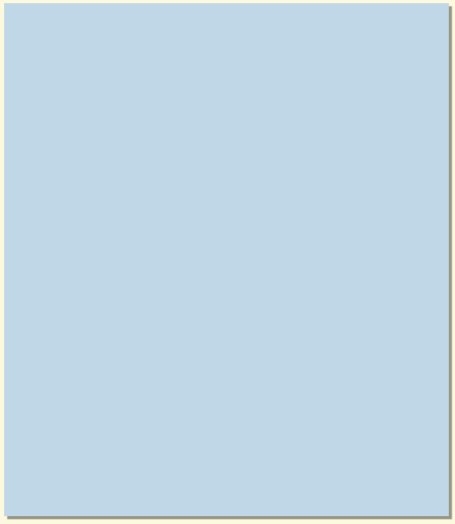


String Comparison



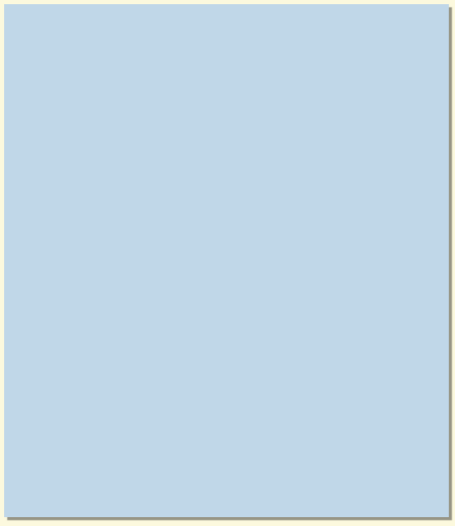
String Comparison

- Relational Operators
  (Comparison)



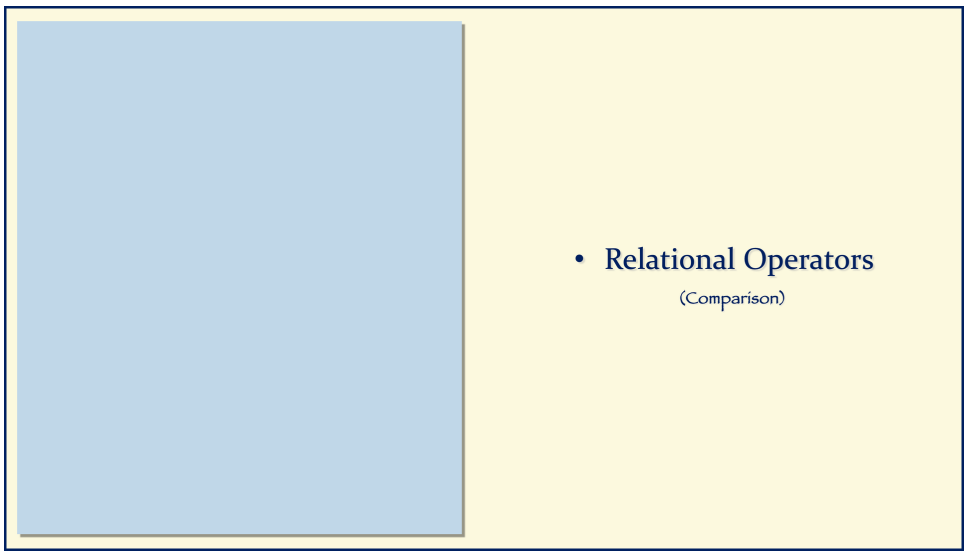
String Comparison

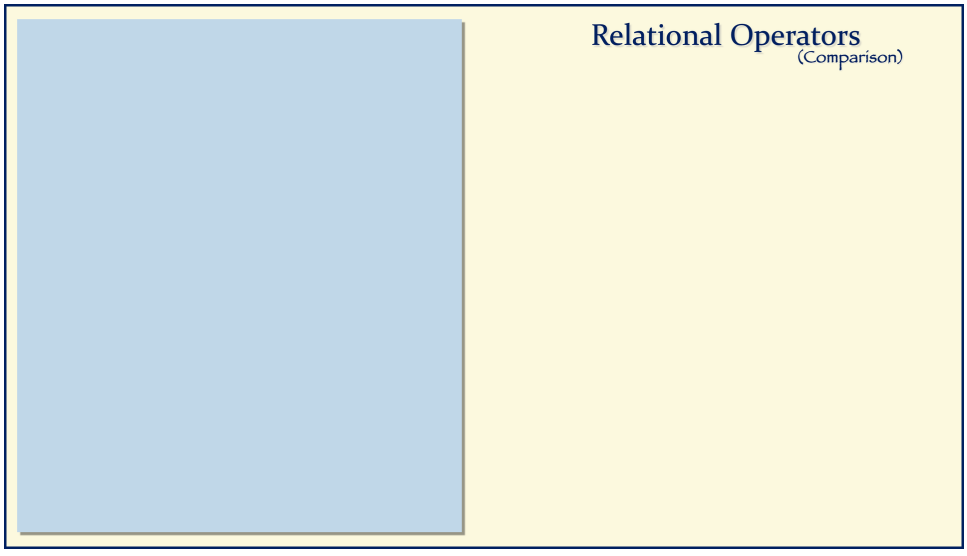
- Relational Operators (Comparison)
- String Comparison

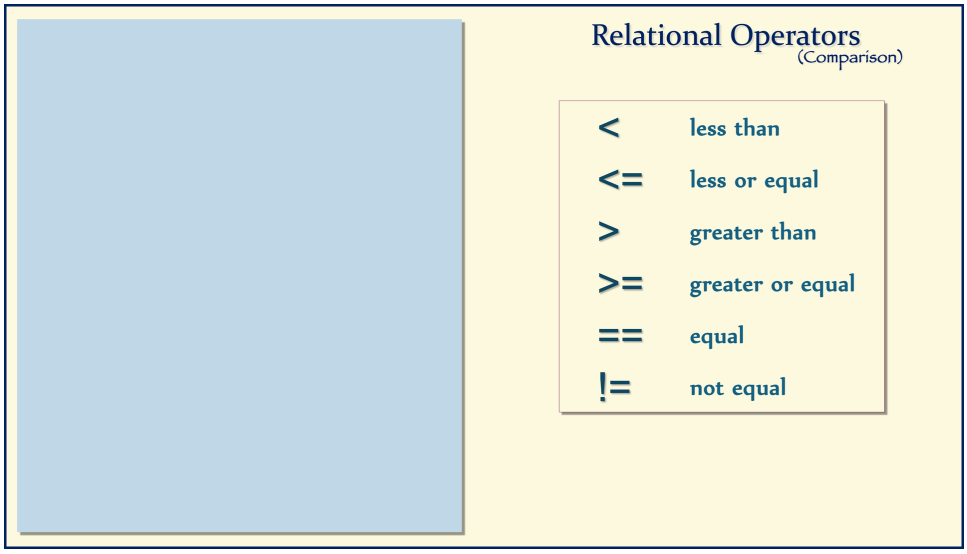


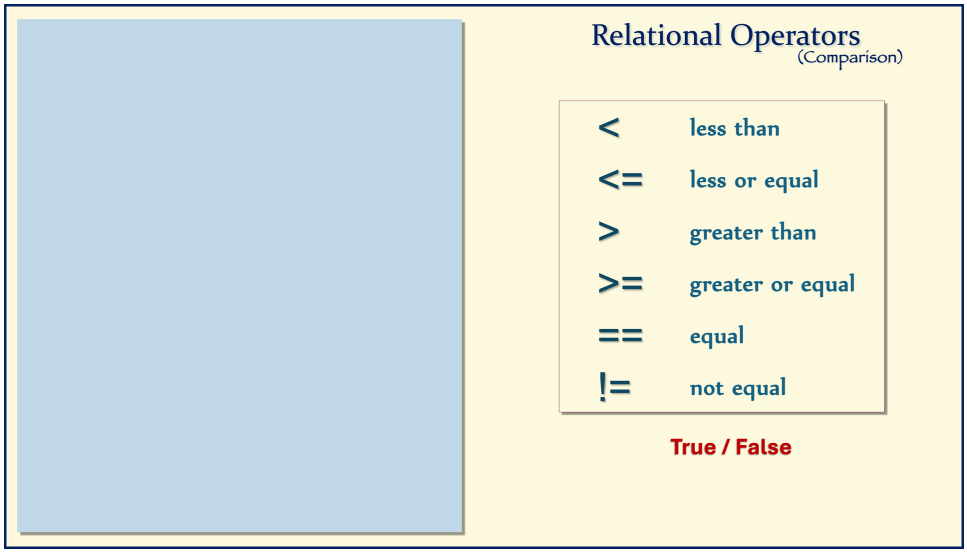
String Comparison

- Relational Operators (Comparison)
- String Comparison
- Relational on All Datatypes

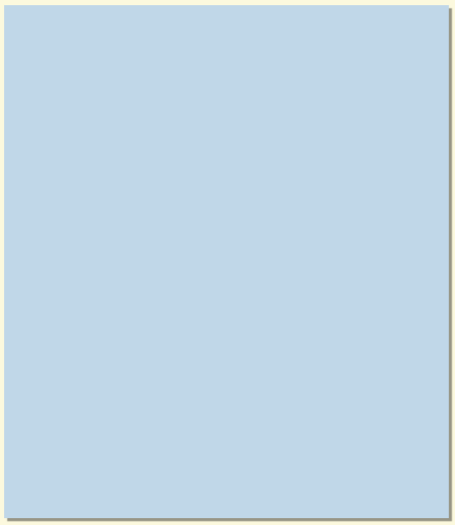

• Relational Operators
(Comparison)


Relational Operators
(Comparison)


Relational Operators
(Comparison)
< less than
<= less or equal
> greater than
>= greater or equal
== equal
!= not equal


Relational Operators
(Comparison)
< less than
<= less or equal
> greater than
>= greater or equal
== equal
!= not equal
True / False

Relational Operators
(Comparison)

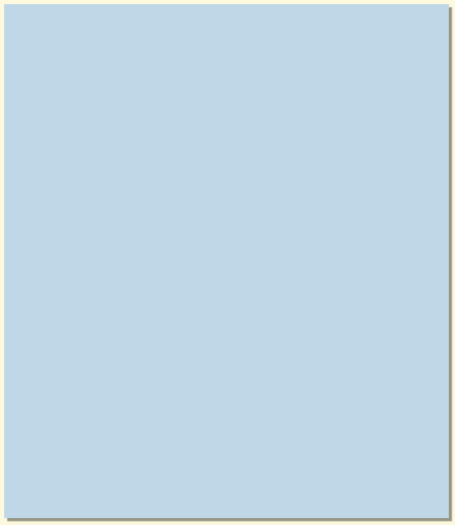


- <
- <=
- >
- >=
- ==
- !=

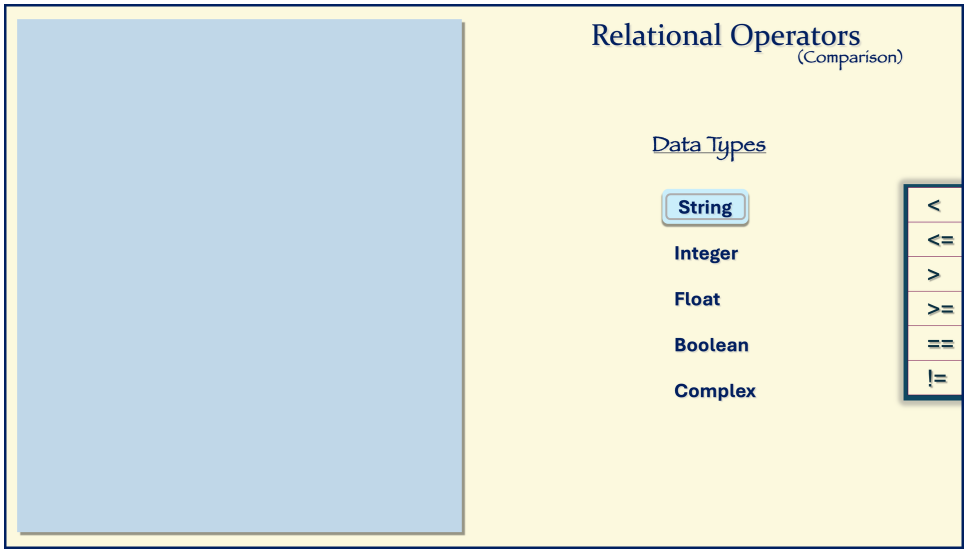
Relational Operators
(Comparison)

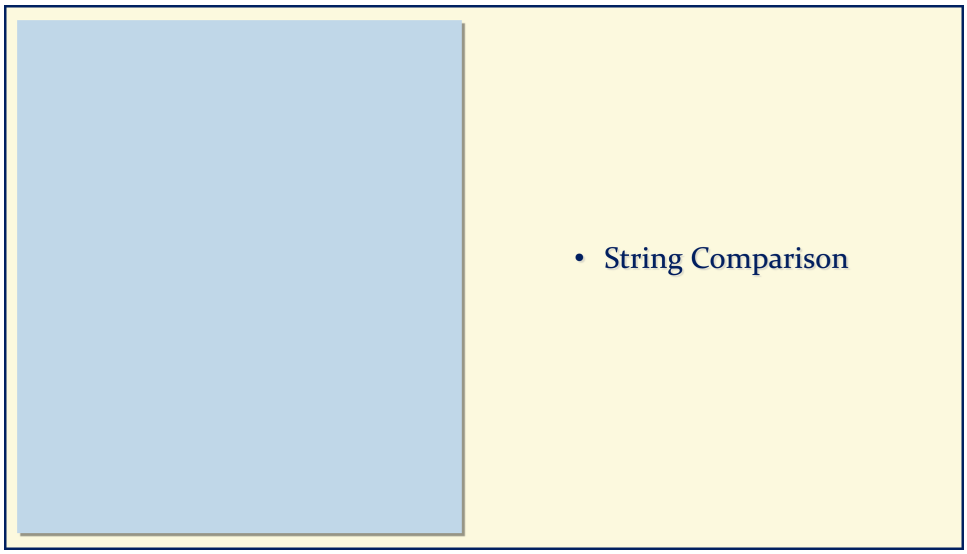
Data Types

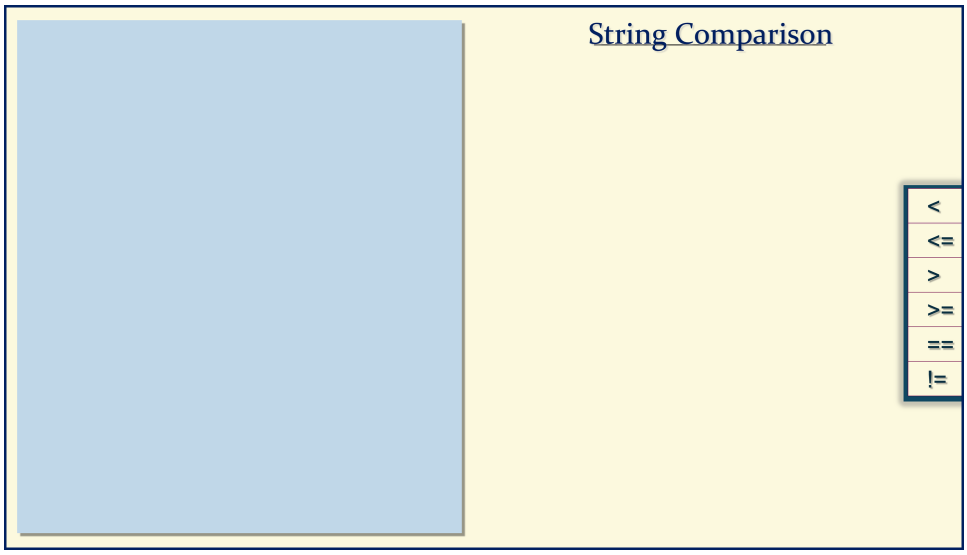
- String
- Integer
- Float
- Boolean
- Complex

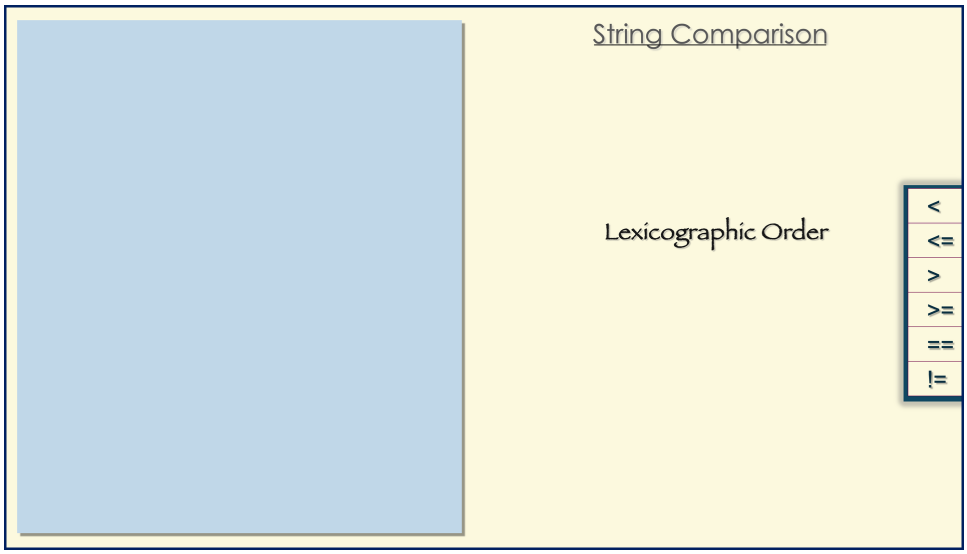


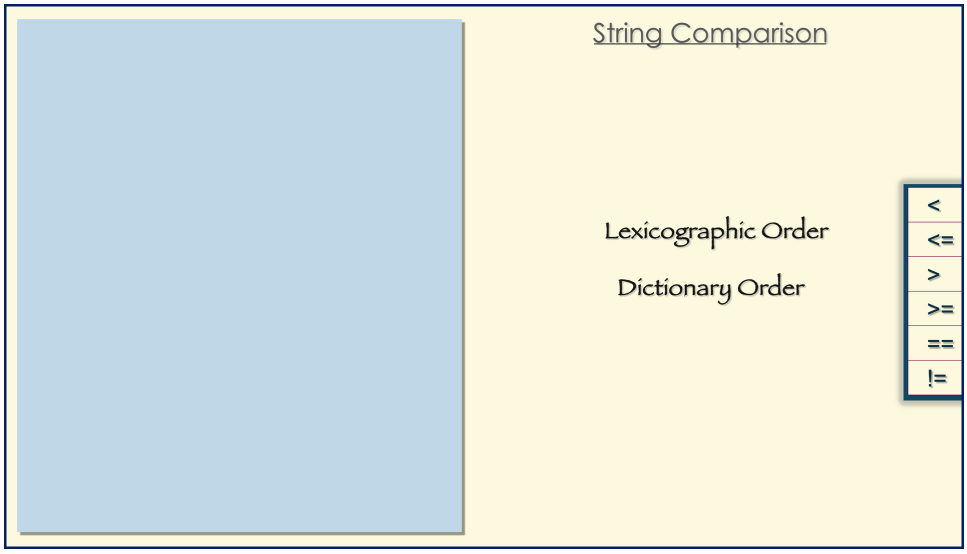
- <
- <=
- >
- >=
- ==
- !=

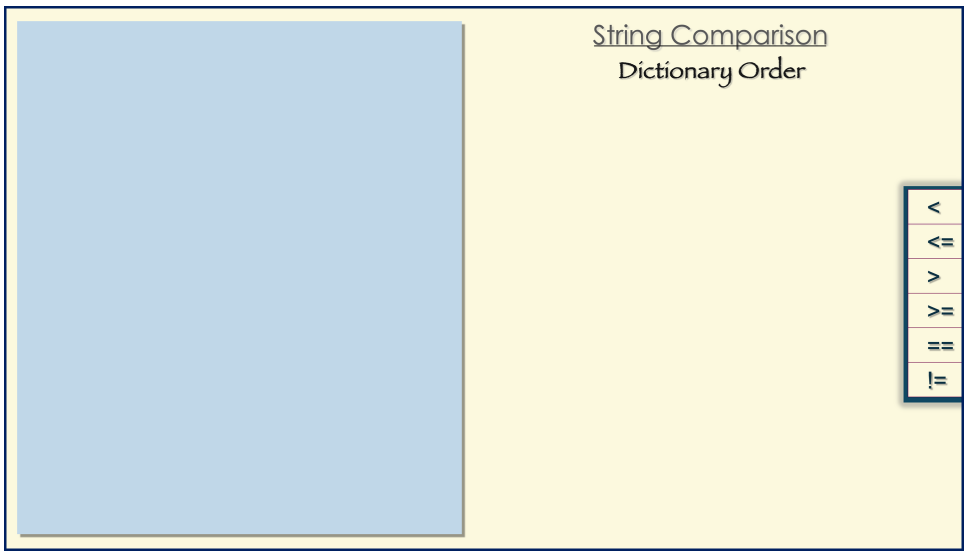

Relational Operators
(Comparison)
Data Types
String
Integer
Float
Boolean
Complex
<
<=
>
>=
==
!=

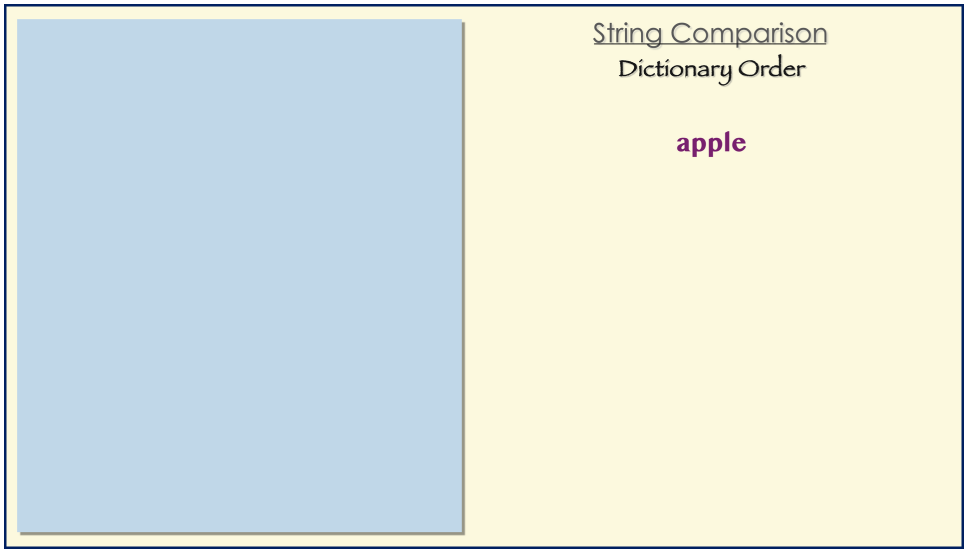

• String Comparison

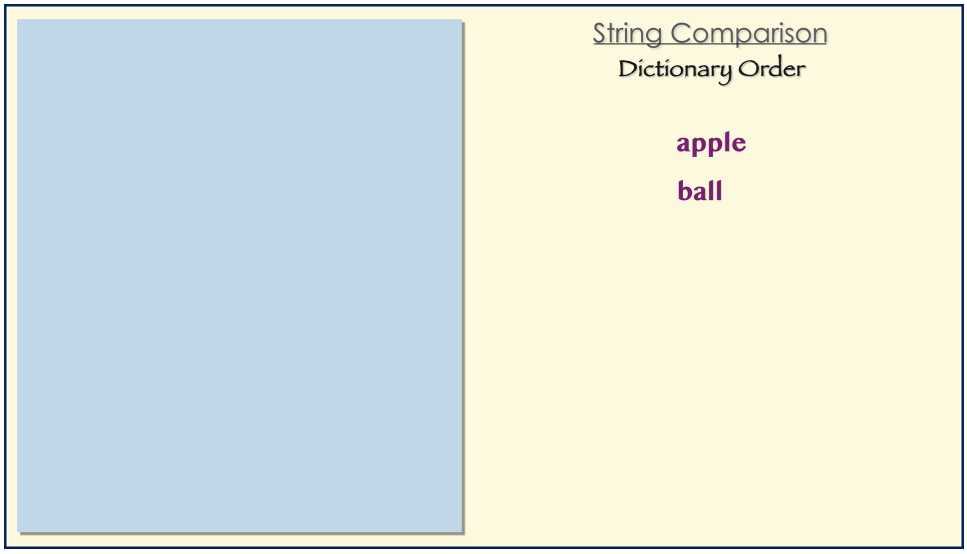

String Comparison
<
<=
>
>=
==
!=

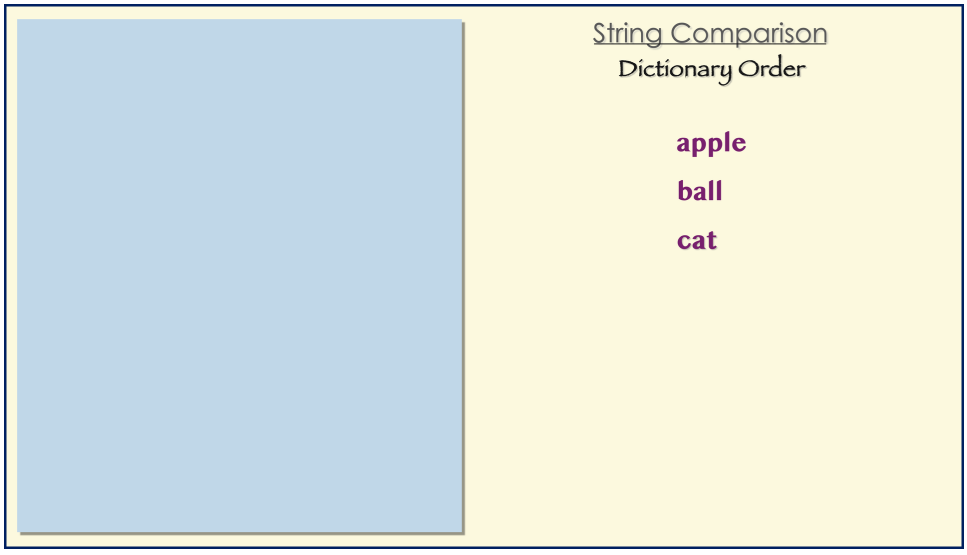

String Comparison
Lexicographic Order
<
<=
>
>=
==
!=

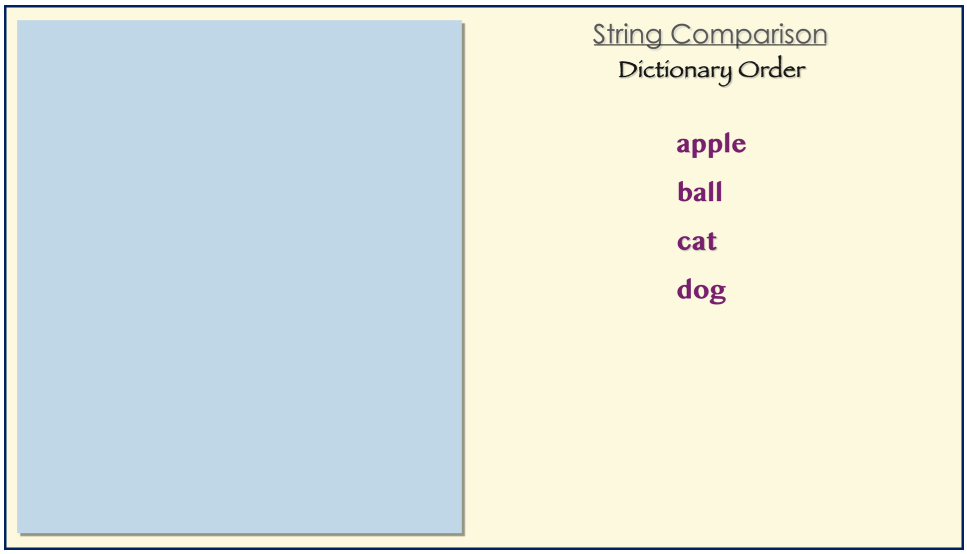

String Comparison
Lexicographic Order
Dictionary Order
<
<=
>
>=
==
!=

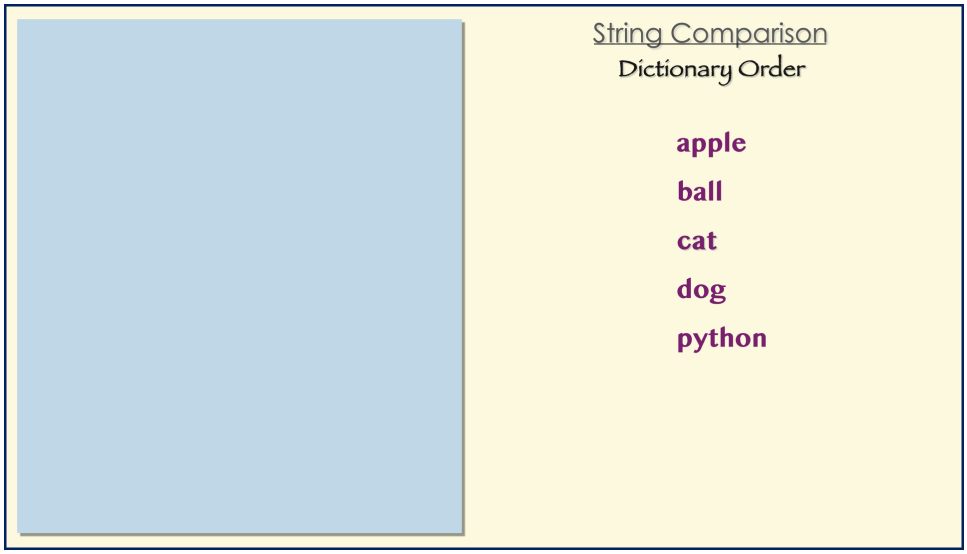

String Comparison
Dictionary Order
<
<=
>
>=
==
!=

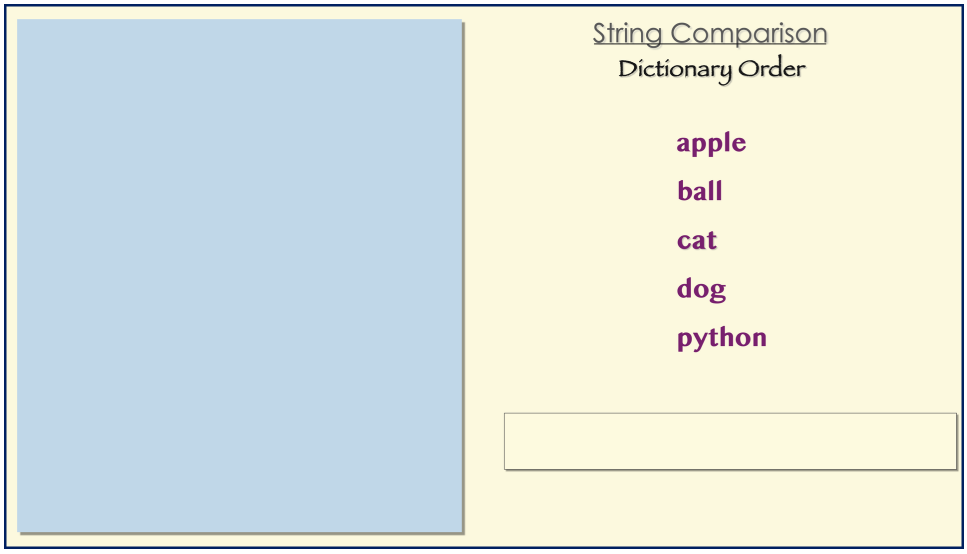

String Comparison
Dictionary Order
apple

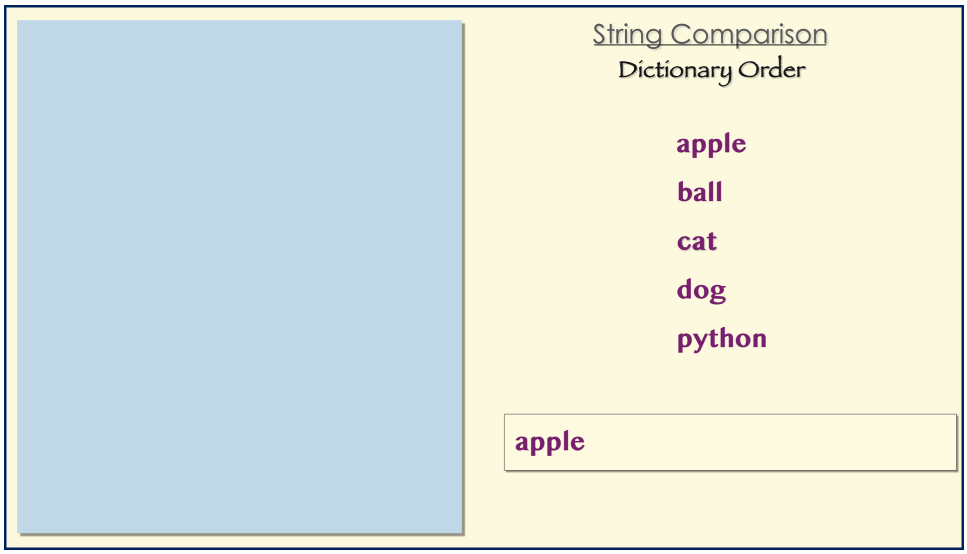

String Comparison
Dictionary Order
apple
ball

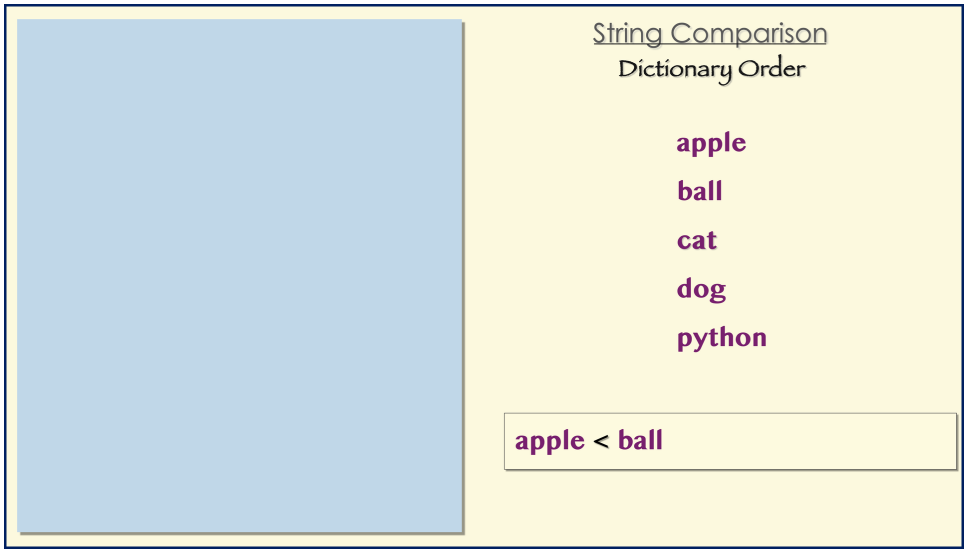

String Comparison
Dictionary Order
apple
ball
cat

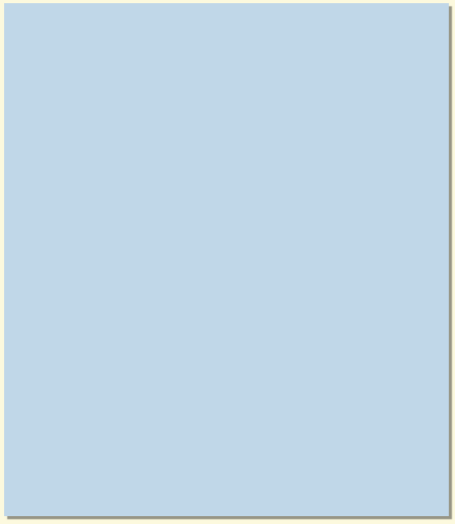

String Comparison
Dictionary Order
apple
ball
cat
dog


String Comparison
Dictionary Order
apple
ball
cat
dog
python


String Comparison
Dictionary Order
apple
ball
cat
dog
python


String Comparison
Dictionary Order
apple
ball
cat
dog
python
apple


String Comparison
Dictionary Order
apple
ball
cat
dog
python
apple < ball



String Comparison

Dictionary Order

apple

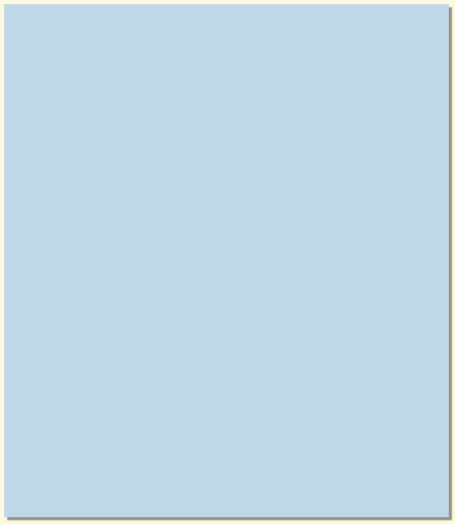
ball

cat

dog

python

apple < ball < cat



String Comparison

Dictionary Order

apple

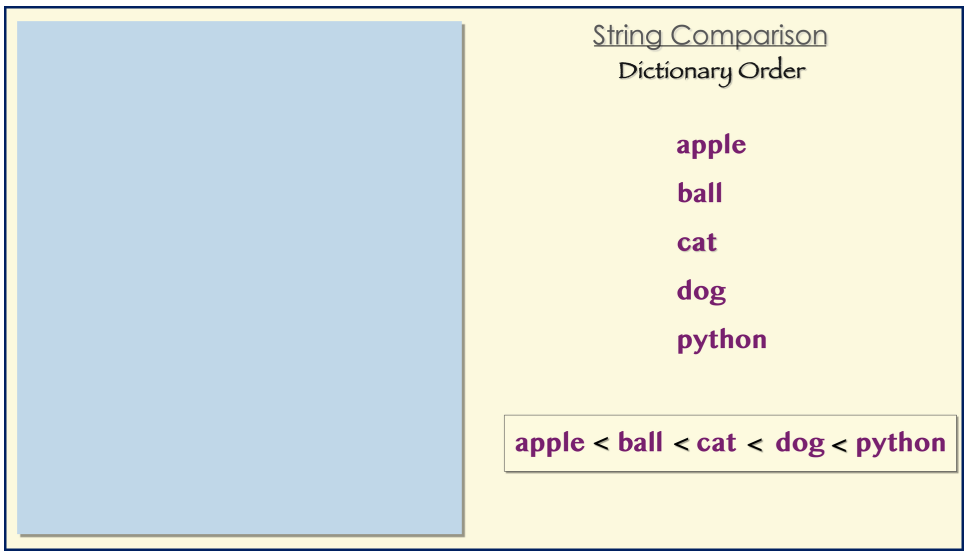
ball

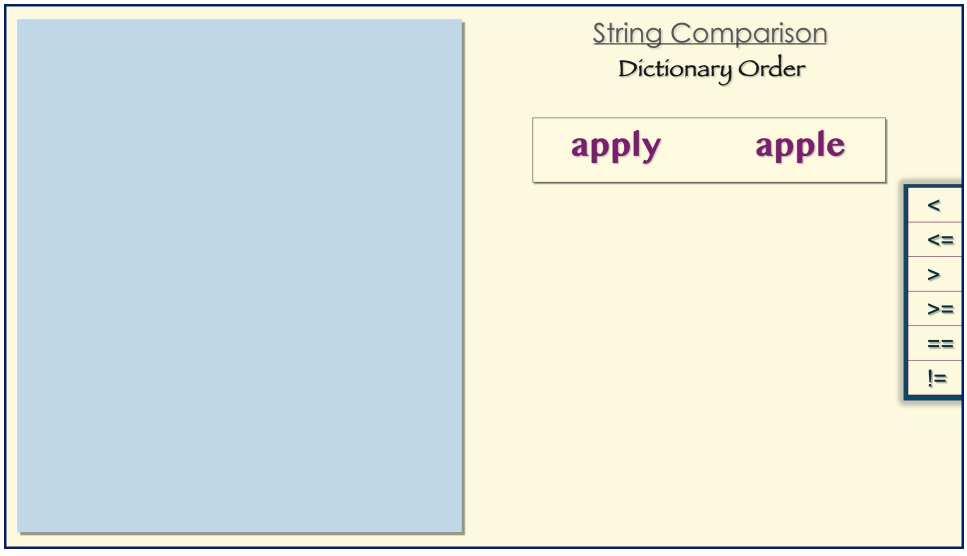
cat

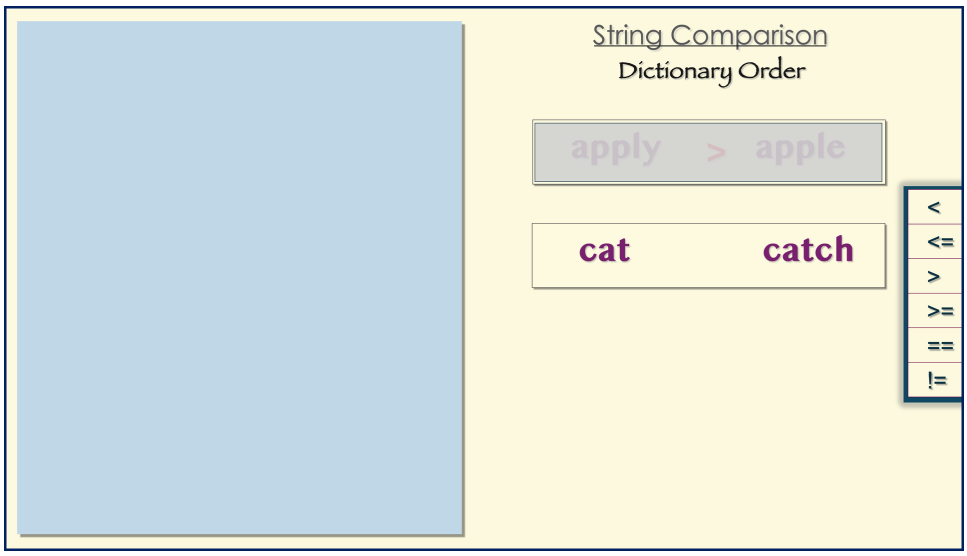
dog

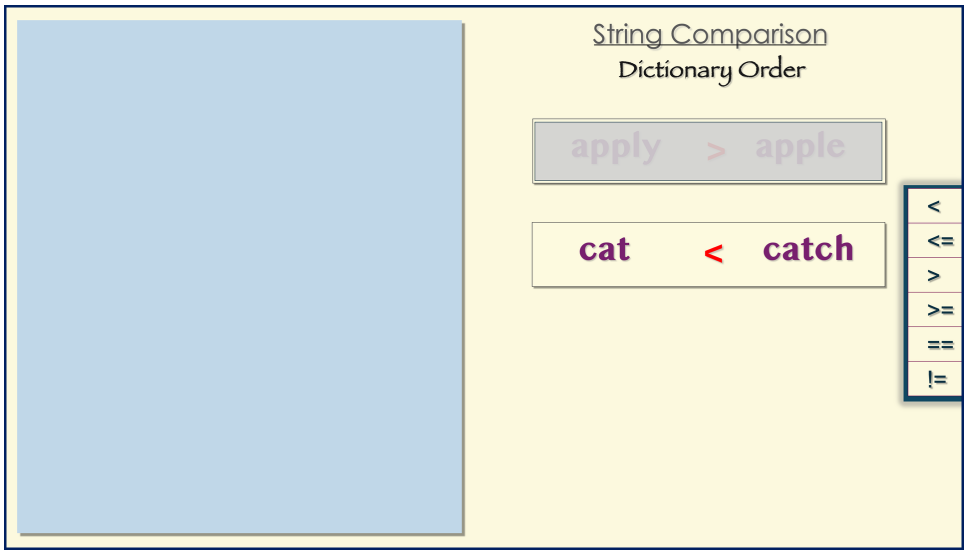
python

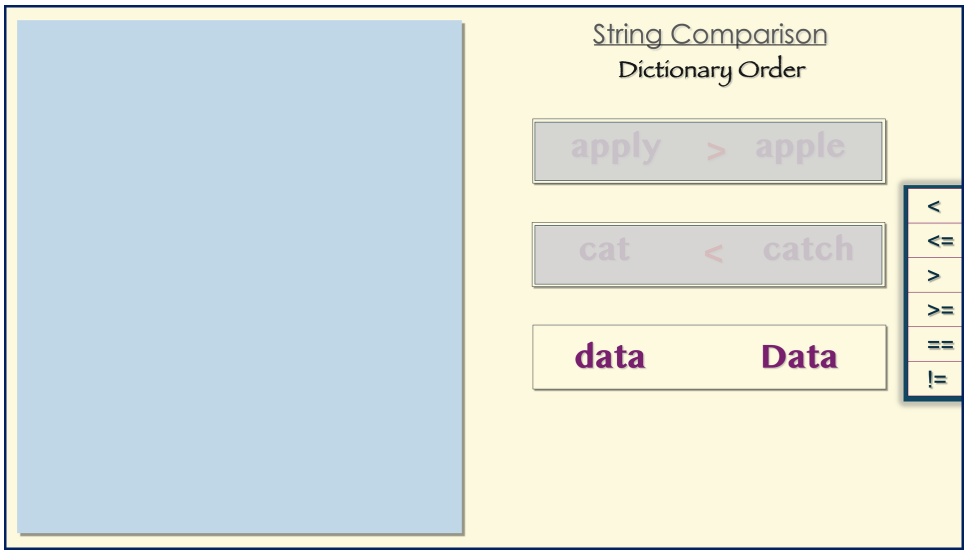
apple < ball < cat < dog

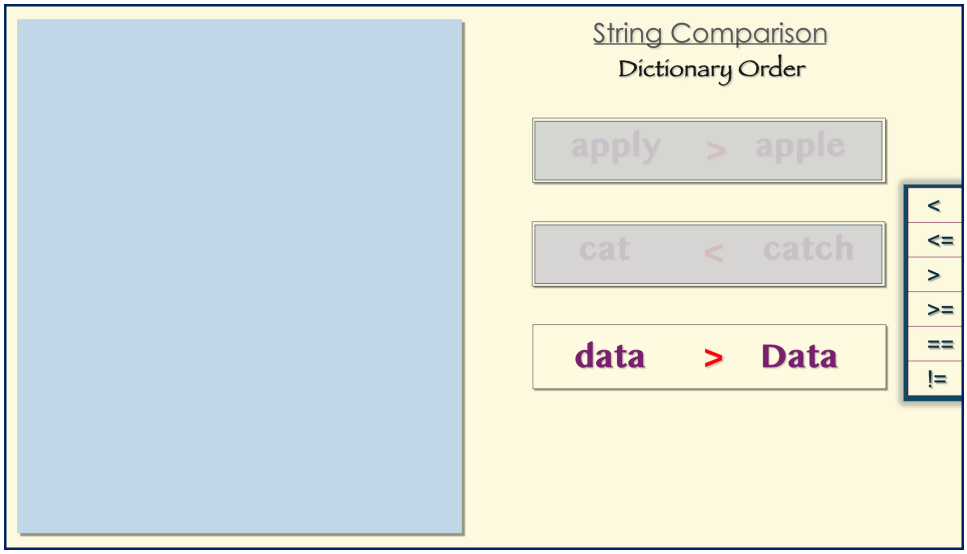

String Comparison
Dictionary Order
apple
ball
cat
dog
python
apple < ball < cat < dog < python

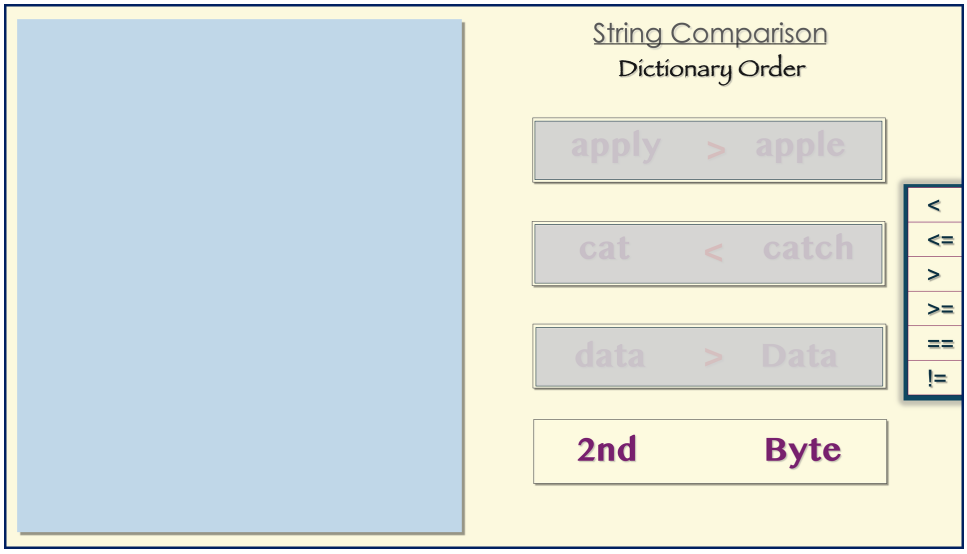

String Comparison
Dictionary Order
apply apple
<
<=
>
>=
==
!=

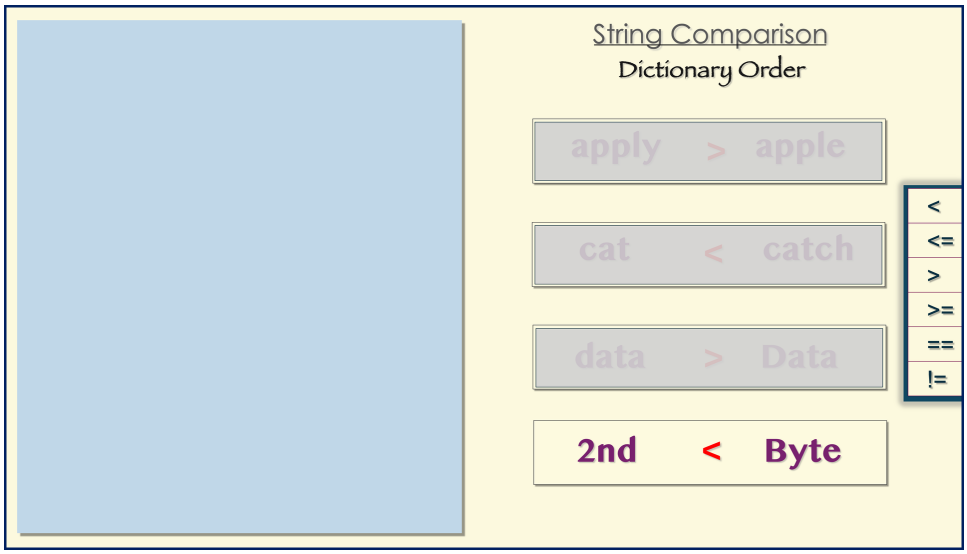

String Comparison
Dictionary Order
apply > apple
cat catch
<
<=
>
>=
==
!=

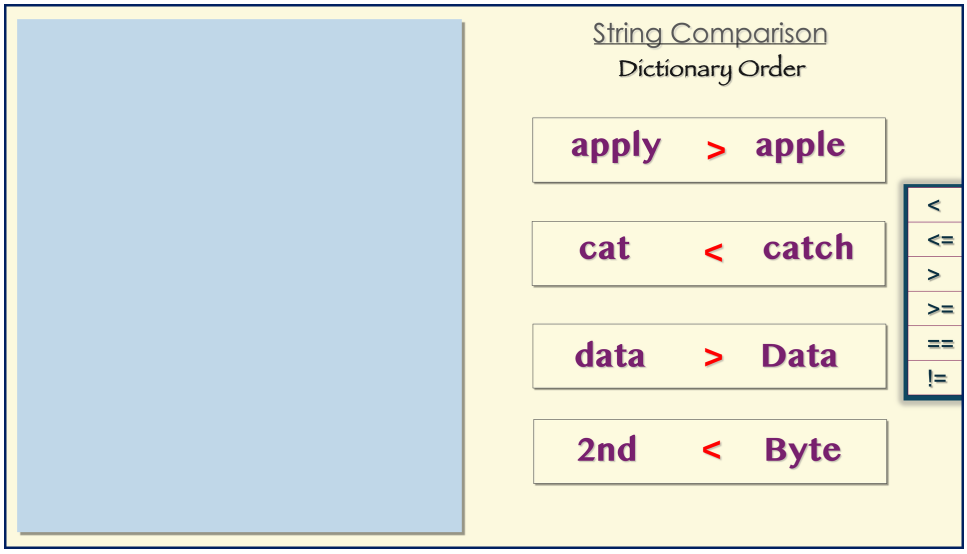

String Comparison
Dictionary Order
apply > apple
cat < catch
<
<=
>
>=
==
!=

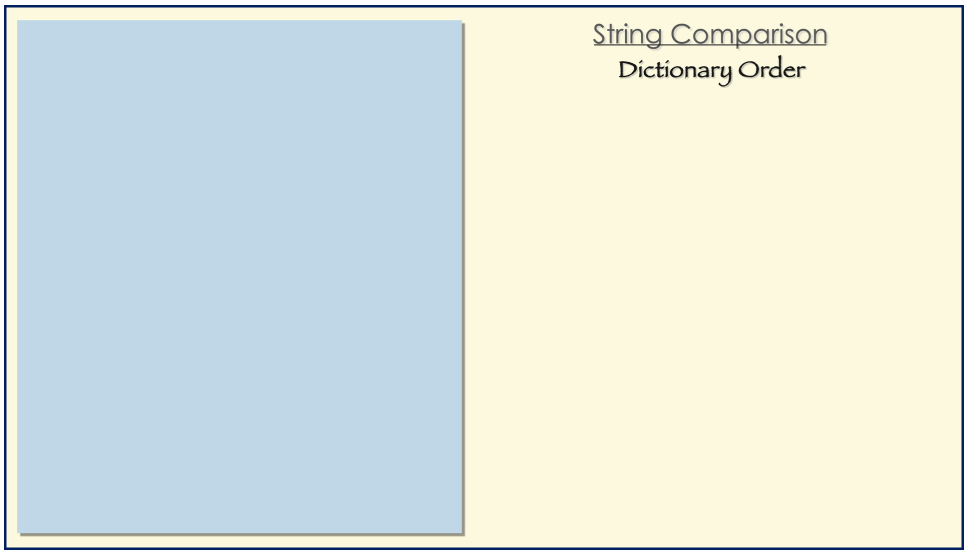

String Comparison
Dictionary Order
apply > apple
cat < catch
data Data
<
<=
>
>=
==
!=

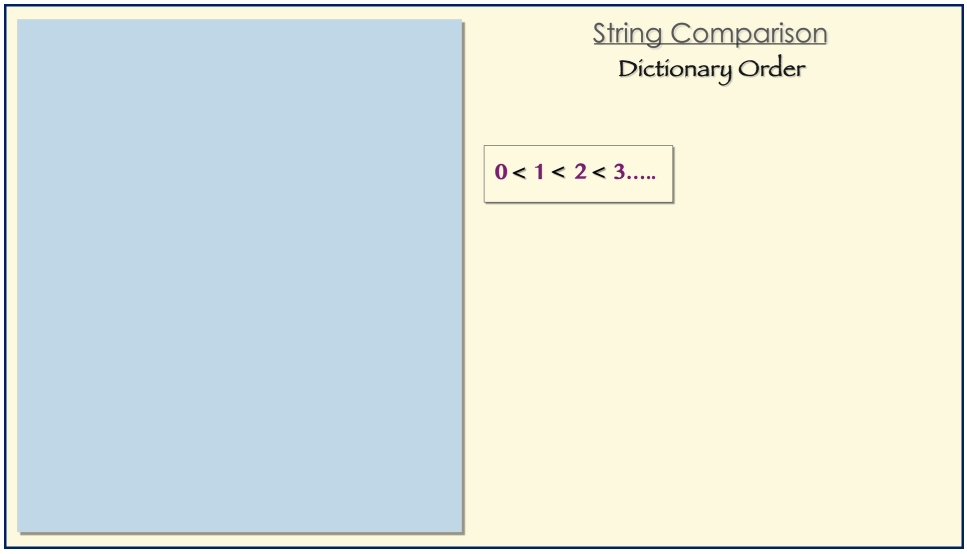

String Comparison
Dictionary Order
apply > apple
cat < catch
data > Data
<
<=
>
>=
==
!=

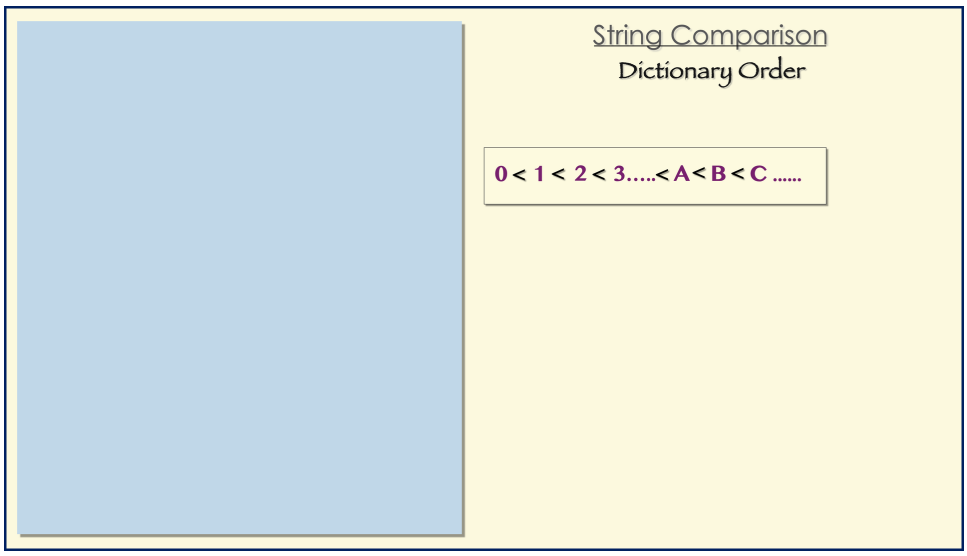

String Comparison
Dictionary Order
apply > apple
cat < catch
data > Data
2nd Byte
<
<=
>
>=
==
!=

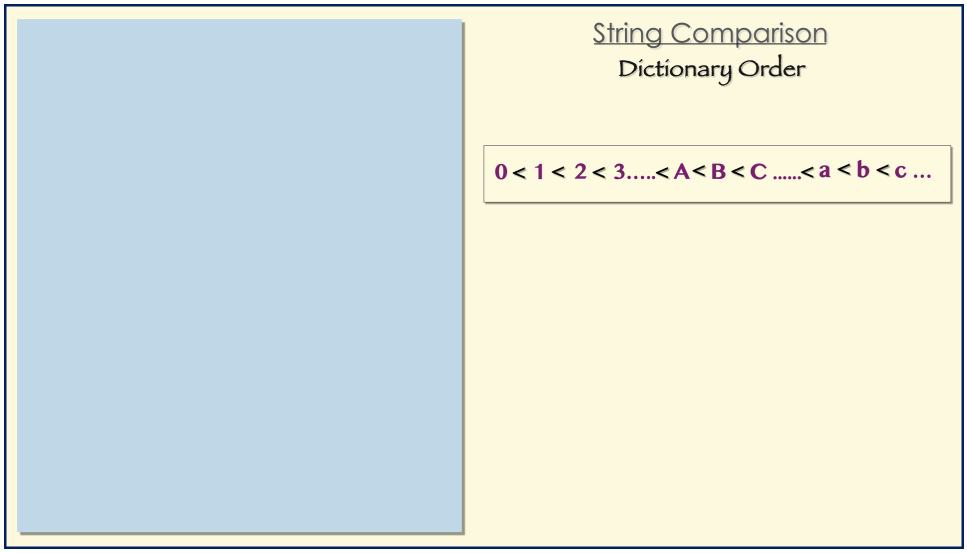

String Comparison
Dictionary Order
apply > apple
cat < catch
data > Data
2nd < Byte
<
<=
>
>=
==
!=

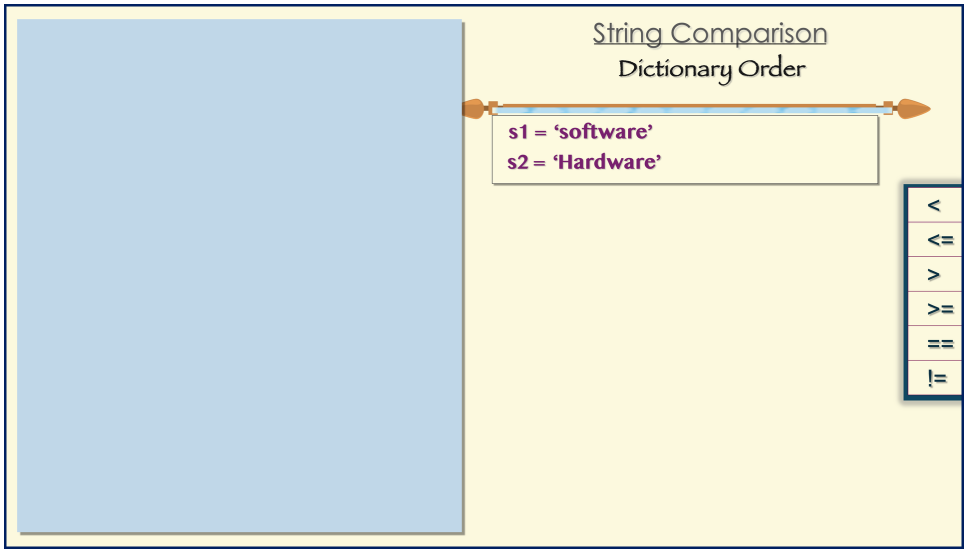

String Comparison
Dictionary Order
apply > apple
cat < catch
data > Data
2nd < Byte
<
<=
>
>=
==
!=

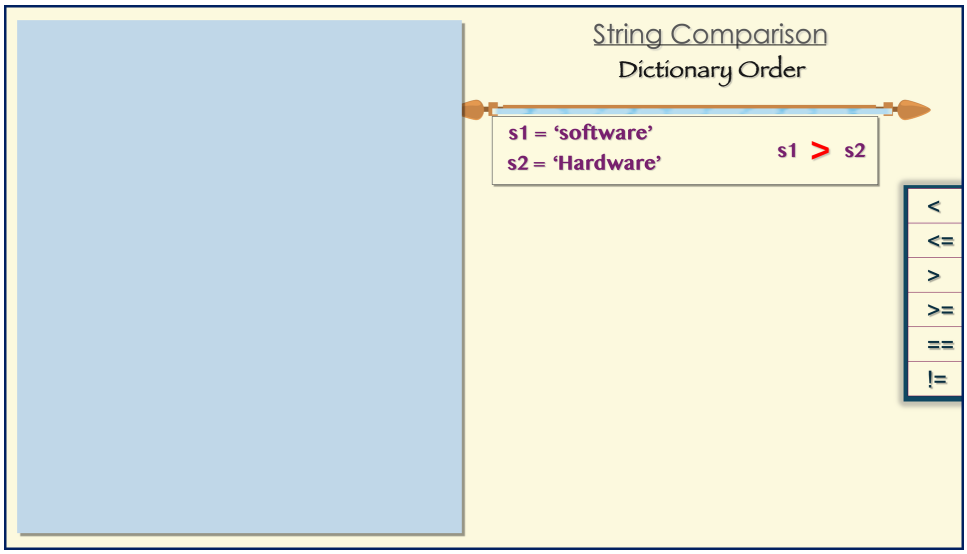

String Comparison
Dictionary Order

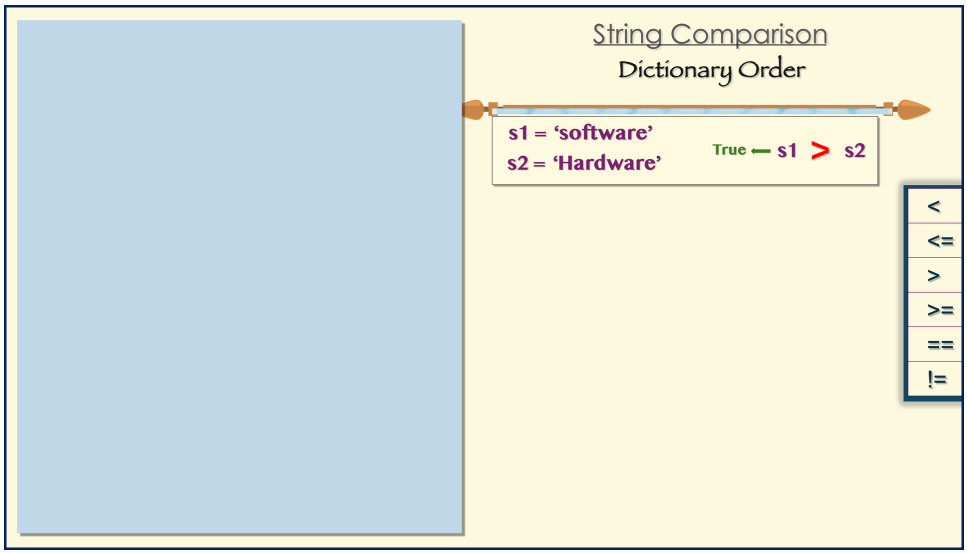

String Comparison
Dictionary Order
0 < 1 < 2 < 3.....

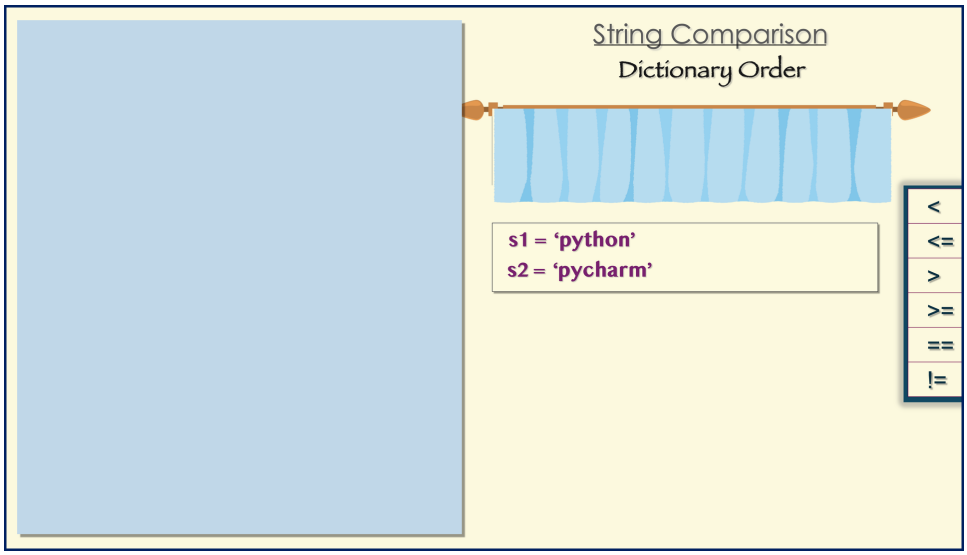

String Comparison
Dictionary Order
0 < 1 < 2 < 3.....< A < B < C .....

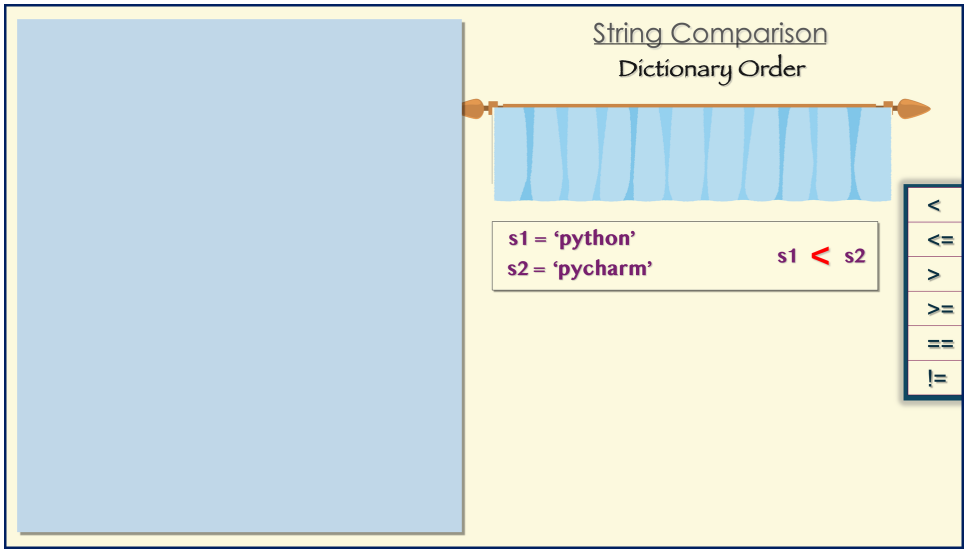

String Comparison
Dictionary Order
0 < 1 < 2 < 3.....< A < B < C .....< a < b < c ...

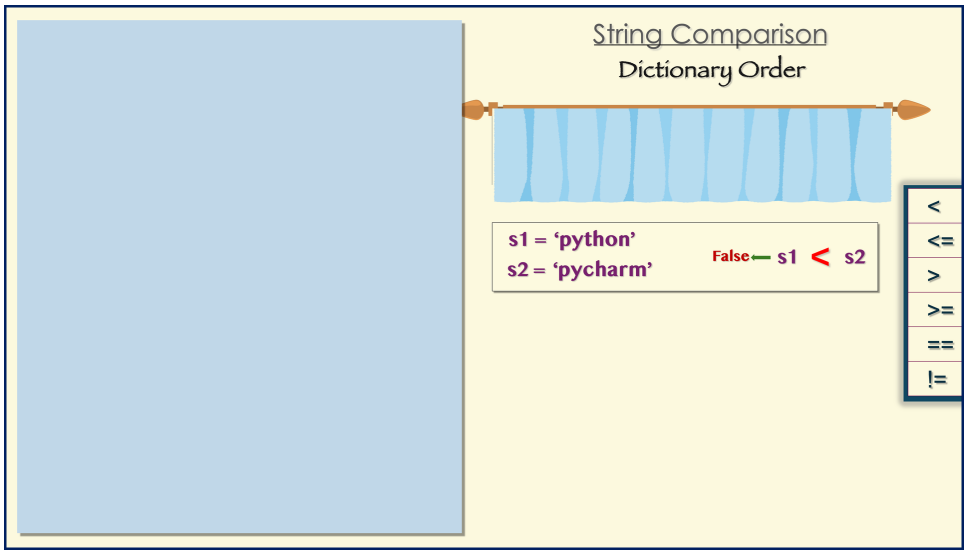

String Comparison
Dictionary Order
s1 = 'software'
s2 = 'Hardware'
<
<=
>
>=
==
!=

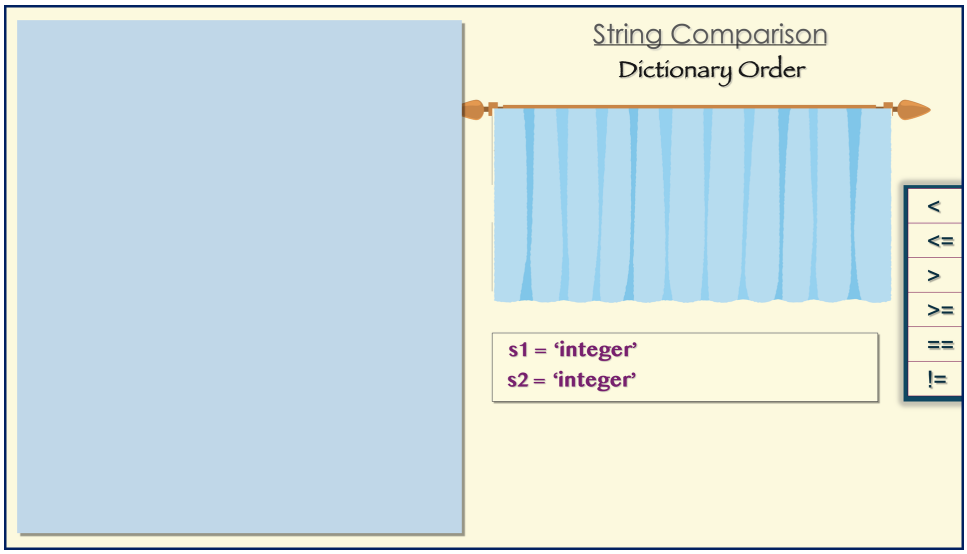

String Comparison
Dictionary Order
s1 = 'software'
s2 = 'Hardware'
s1 > s2
<
<=
>
>=
==
!=

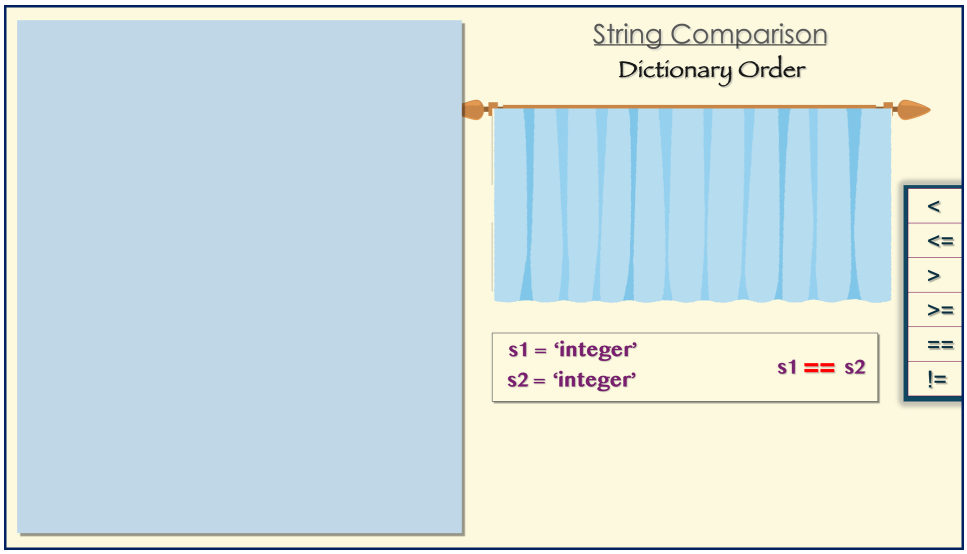

String Comparison
Dictionary Order
s1 = 'software'
s2 = 'Hardware'
True ⟵ s1 > s2
<
<=
>
>=
==
!=

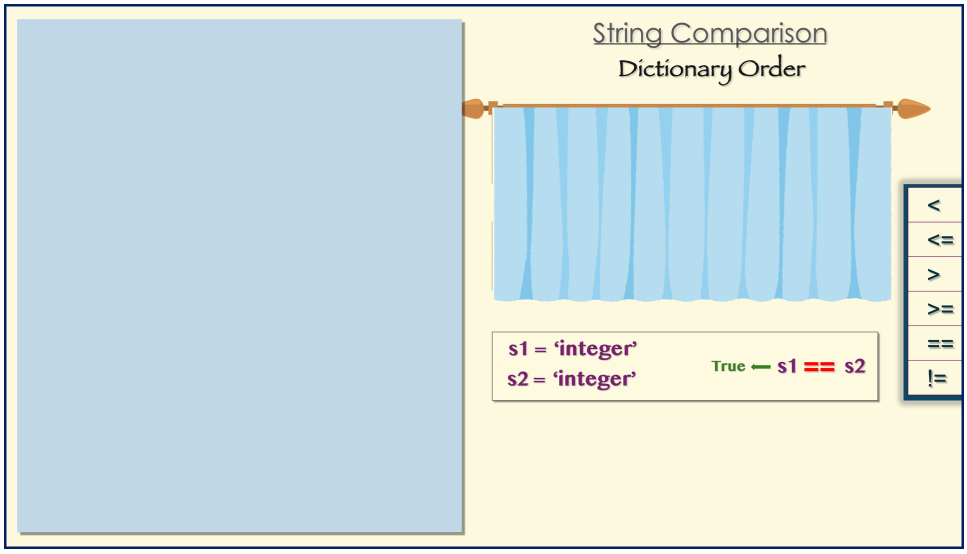

String Comparison
Dictionary Order
s1 = 'python'
s2 = 'pycharm'
<
<=
>
>=
==
!=

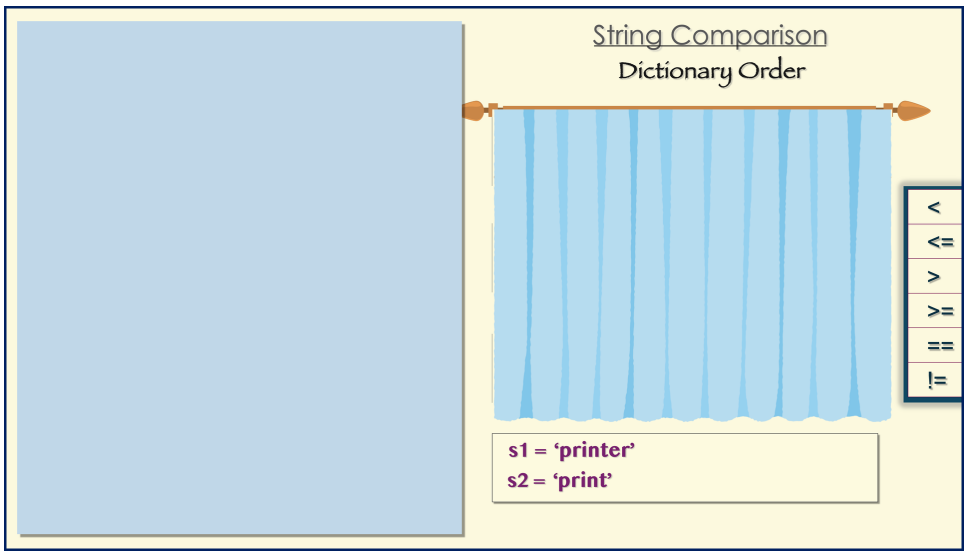

String Comparison
Dictionary Order
s1 = 'python'
s2 = 'pycharm'
s1 < s2
<
<=
>
>=
==
!=

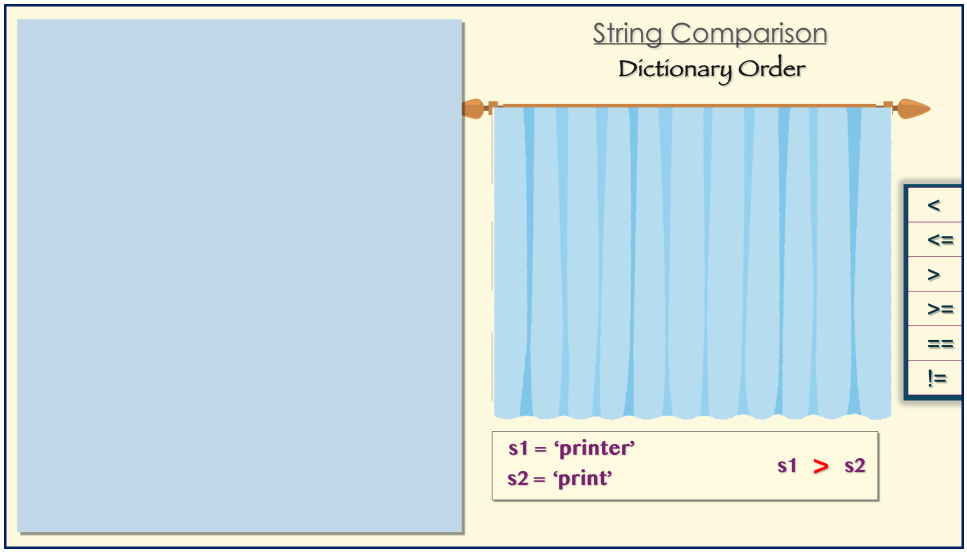

String Comparison
Dictionary Order
s1 = 'python'
s2 = 'pycharm'
False
s1 < s2
<
<=
>
>=
==
!=

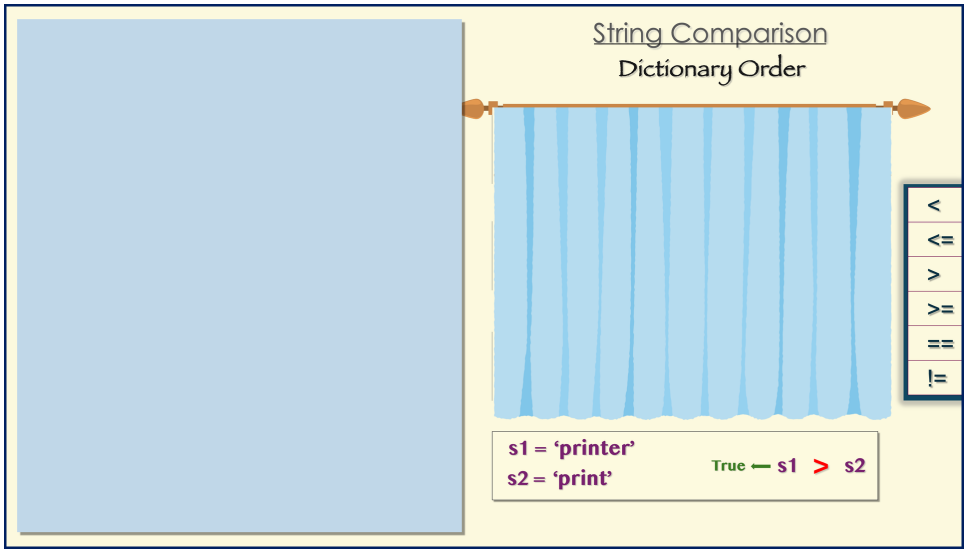

String Comparison
Dictionary Order
s1 = 'integer'
s2 = 'integer'
<
<=
>
>=
==
!=

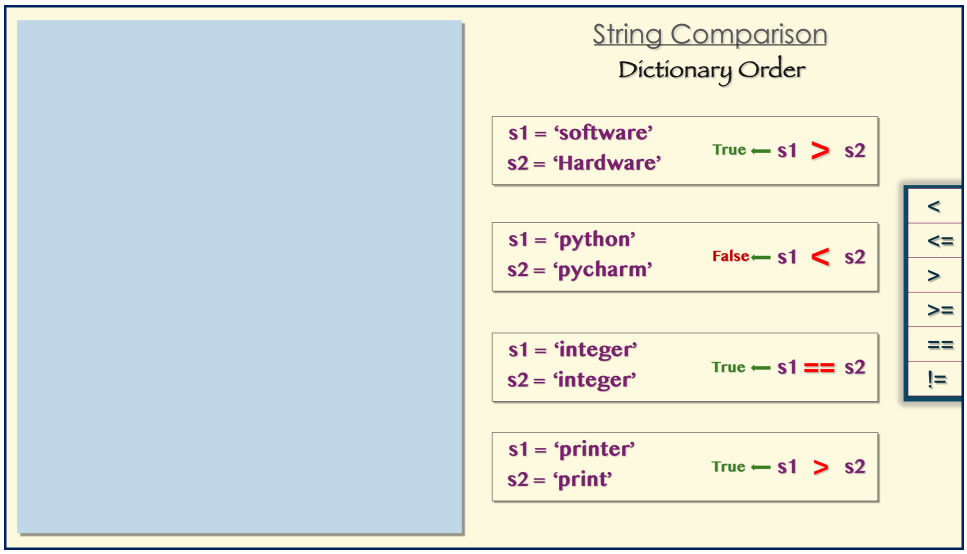

String Comparison
Dictionary Order
s1 = 'integer'
s2 = 'integer'
s1 == s2
<
<=
>
>=
==
!=

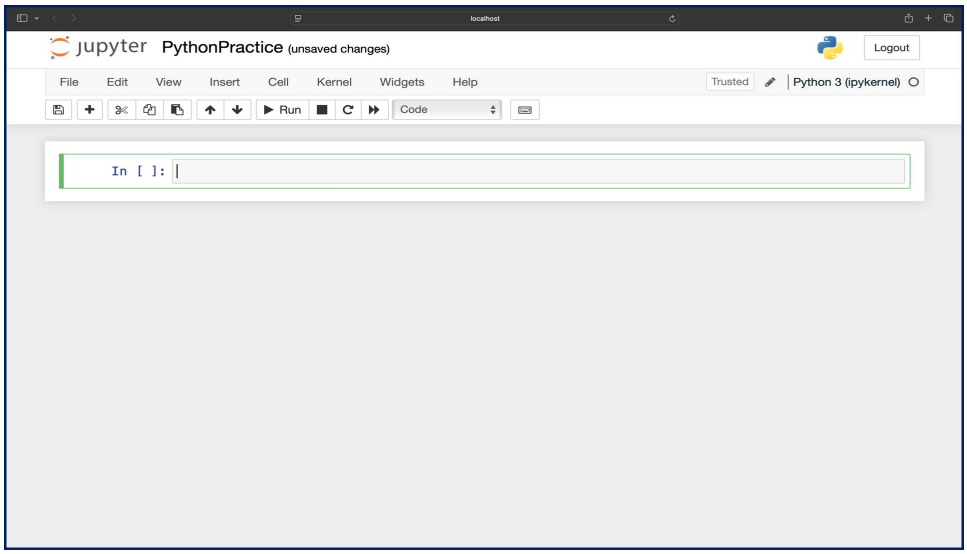

String Comparison
Dictionary Order
s1 = 'integer'
s2 = 'integer'
True ⟵ s1 == s2
<
<=
>
>=
==
!=

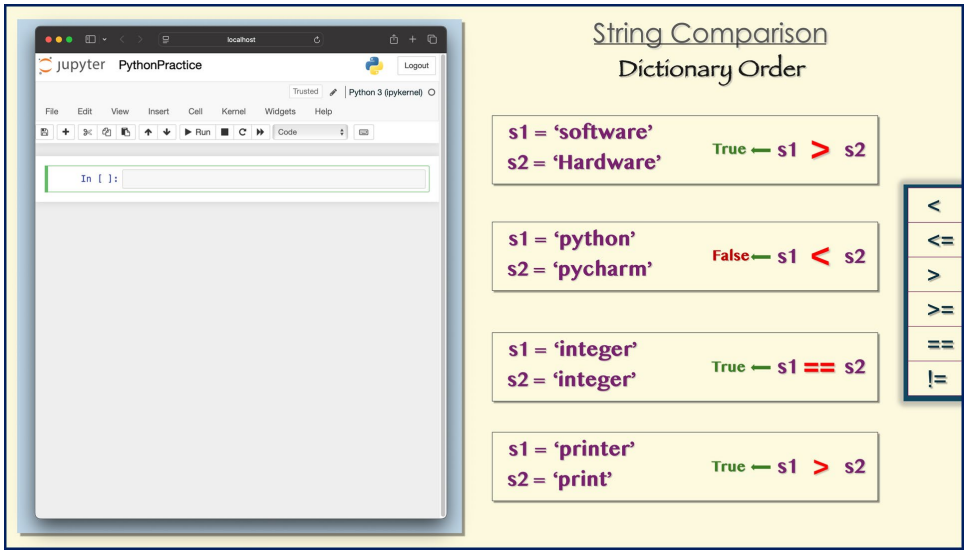

String Comparison
Dictionary Order
<
<=
>
>=
==
!=
s1 = 'printer'
s2 = 'print'

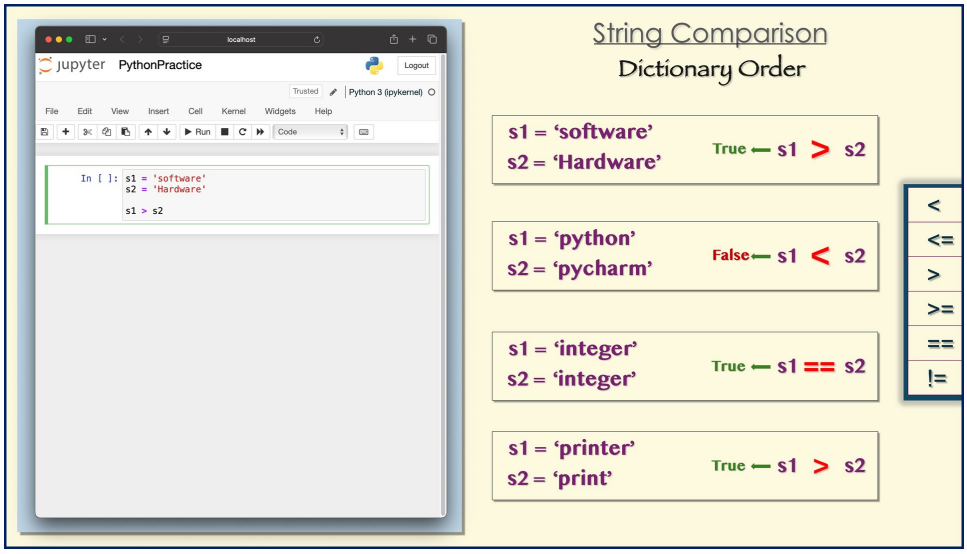

String Comparison
Dictionary Order
<
<=
>
>=
==
!=
s1 = 'printer'
s2 = 'print'
s1 > s2

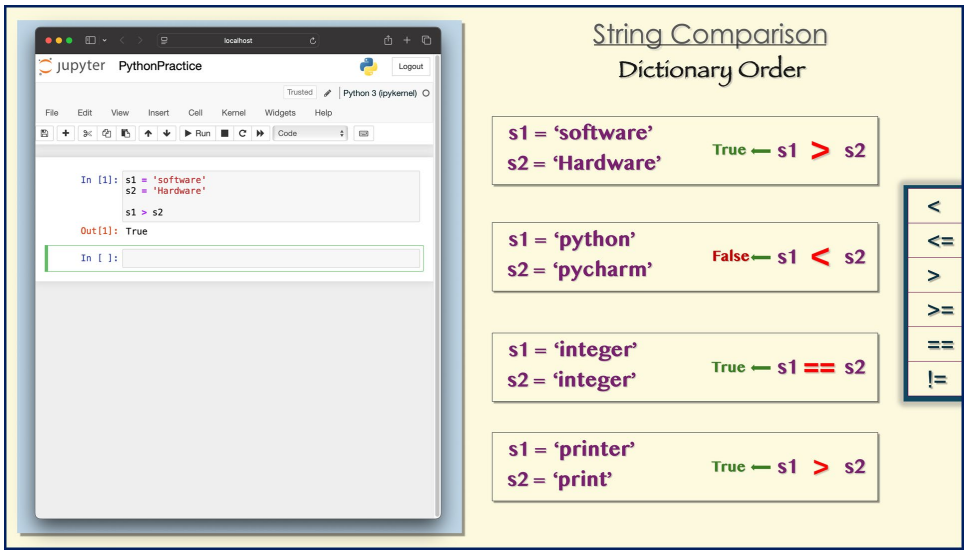

String Comparison
Dictionary Order
<
<=
>
>=
==
!=
s1 = 'printer'
s2 = 'print'
True ⟵ s1 > s2

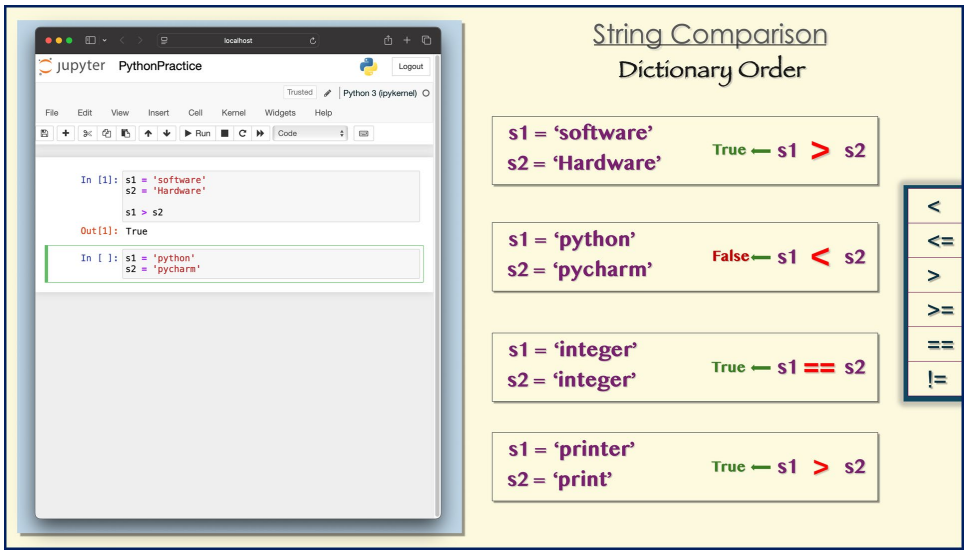

String Comparison
Dictionary Order
s1 = 'software'
s2 = 'Hardware'
True ⟵ s1 > s2
s1 = 'python'
s2 = 'pycharm'
False⟵ s1 < s2
s1 = 'integer'
s2 = 'integer'
True ⟵ s1 == s2
s1 = 'printer'
s2 = 'print'
True ⟵ s1 > s2
<
<=
>
>=
==
!=

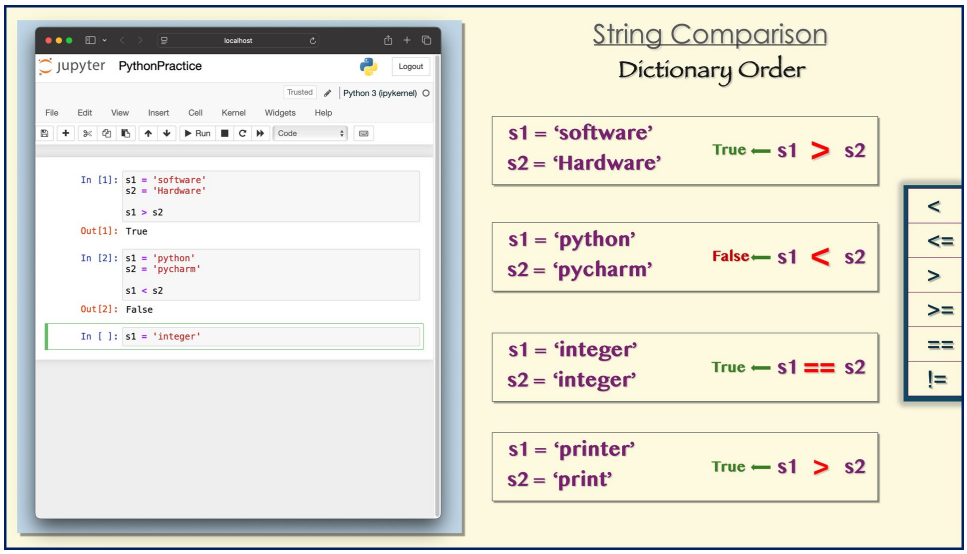

localhost
jupyter PythonPractice (unsaved changes)
Logout
File Edit View Insert Cell Kernel Widgets Help
Trusted
Python 3 (ipykernel)
Run
Code
In [ ]:

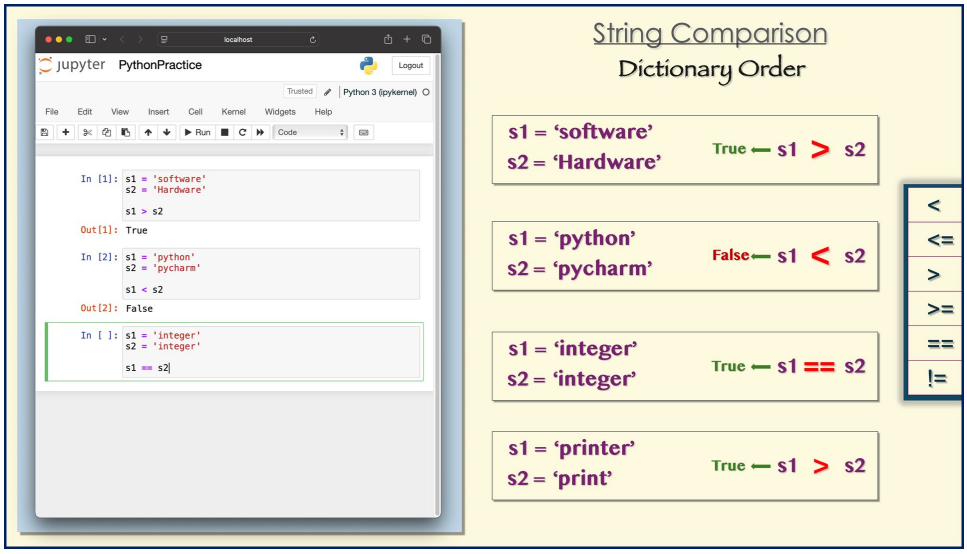

localhost
jupyter PythonPractice
Logout
Trusted
Python 3 (ipykernel)
File Edit View Insert Cell Kernel Widgets Help
Run
Code
In [ ]:
String Comparison
Dictionary Order
s1 = 'software'
s2 = 'Hardware'
True ⟵ s1 > s2
s1 = 'python'
s2 = 'pycharm'
False ⟵ s1 < s2
s1 = 'integer'
s2 = 'integer'
True ⟵ s1 == s2
s1 = 'printer'
s2 = 'print'
True ⟵ s1 > s2
<
<=
>
>=
==
!=

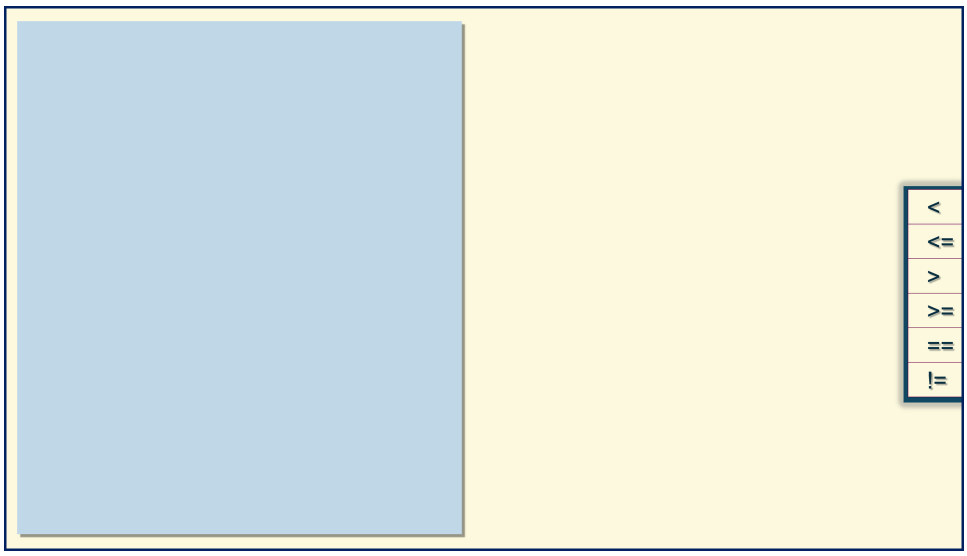

localhost
jupyter PythonPractice
Logout
Trusted
Python 3 (ipykernel)
File Edit View Insert Cell Kernel Widgets Help
Run
Code
In [ ]:
s1 = 'software'
s2 = 'Hardware'
s1 > s2
String Comparison
Dictionary Order
s1 = 'software'
s2 = 'Hardware'
True ⟵ s1 > s2
s1 = 'python'
s2 = 'pycharm'
False ⟵ s1 < s2
s1 = 'integer'
s2 = 'integer'
True ⟵ s1 == s2
s1 = 'printer'
s2 = 'print'
True ⟵ s1 > s2
<
<=
>
>=
==
!=

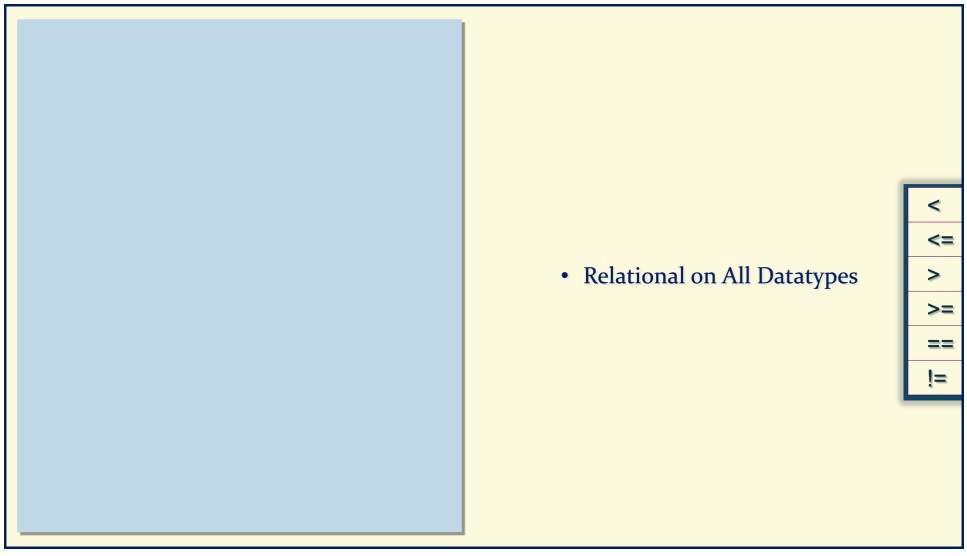

localhost
jupyter PythonPractice
Logout
Trusted
Python 3 (pykernel)
File Edit View Insert Cell Kernel Widgets Help
Run
Code
In [1]: s1 = 'software'
s2 = 'Hardware'
s1 > s2
Out[1]: True
In [ ]:
String Comparison
Dictionary Order
s1 = 'software'
s2 = 'Hardware'
True ⟵ s1 > s2
s1 = 'python'
s2 = 'pycharm'
False ⟵ s1 < s2
s1 = 'integer'
s2 = 'integer'
True ⟵ s1 == s2
s1 = 'printer'
s2 = 'print'
True ⟵ s1 > s2
<
<=
>
>=
==
!=

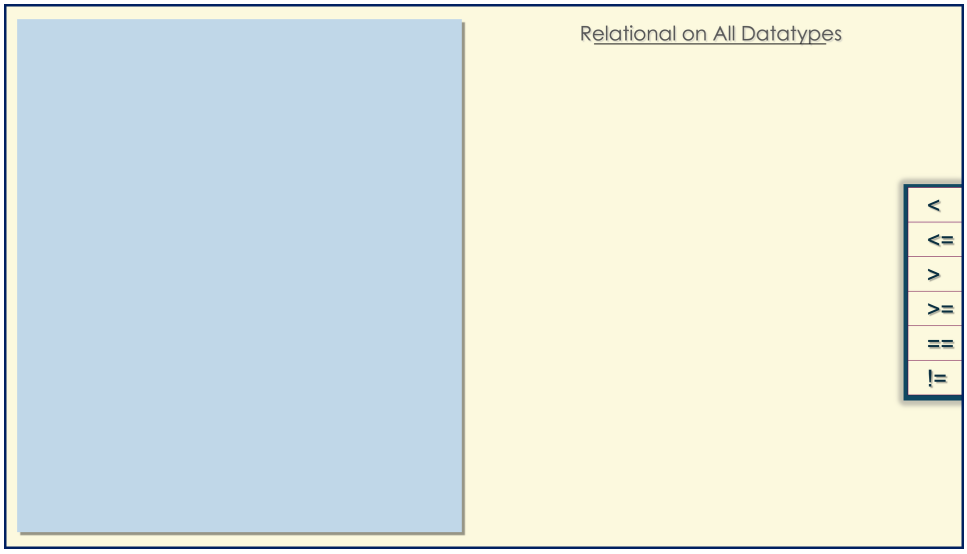

localhost
jupyter PythonPractice
Logout
Trusted
Python 3 (pykernel)
File Edit View Insert Cell Kernel Widgets Help
Run
Code
In [1]: s1 = 'software'
s2 = 'Hardware'
s1 > s2
Out[1]: True
In [ ]: s1 = 'python'
s2 = 'pycharm'
String Comparison
Dictionary Order
s1 = 'software'
s2 = 'Hardware'
True ⟵ s1 > s2
s1 = 'python'
s2 = 'pycharm'
False ⟵ s1 < s2
s1 = 'integer'
s2 = 'integer'
True ⟵ s1 == s2
s1 = 'printer'
s2 = 'print'
True ⟵ s1 > s2
<
<=
>
>=
==
!=

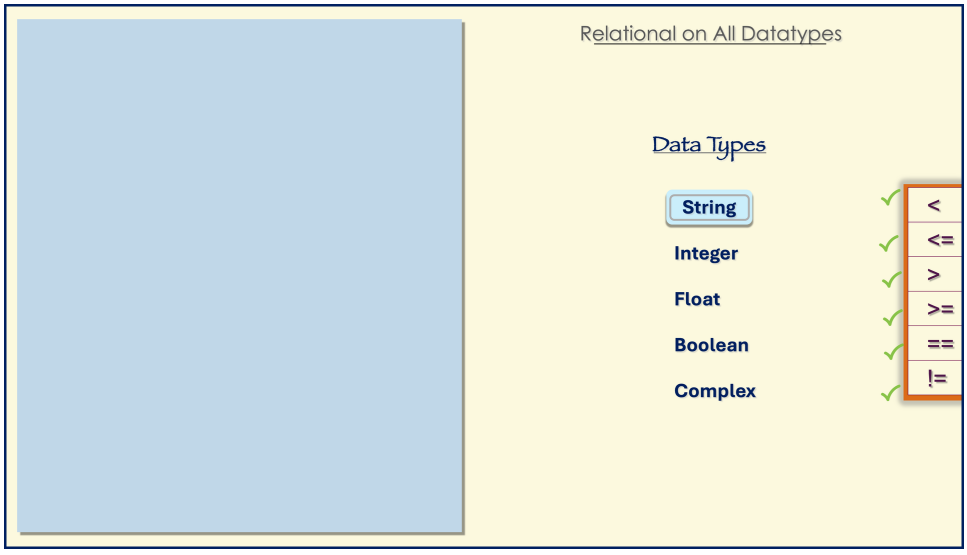

localhost
jupyter PythonPractice
Logout
Trusted
Python 3 (ipykernel)
File Edit View Insert Cell Kernel Widgets Help
Run
Code
In [1]: s1 = 'software'
s2 = 'Hardware'
s1 > s2
Out[1]: True
In [2]: s1 = 'python'
s2 = 'pycharm'
s1 < s2
Out[2]: False
In [ ]: s1 = 'integer'
String Comparison
Dictionary Order
s1 = 'software'
s2 = 'Hardware'
True ⟵ s1 > s2
s1 = 'python'
s2 = 'pycharm'
False ⟵ s1 < s2
s1 = 'integer'
s2 = 'integer'
True ⟵ s1 == s2
s1 = 'printer'
s2 = 'print'
True ⟵ s1 > s2
<
<=
>
>=
==
!=

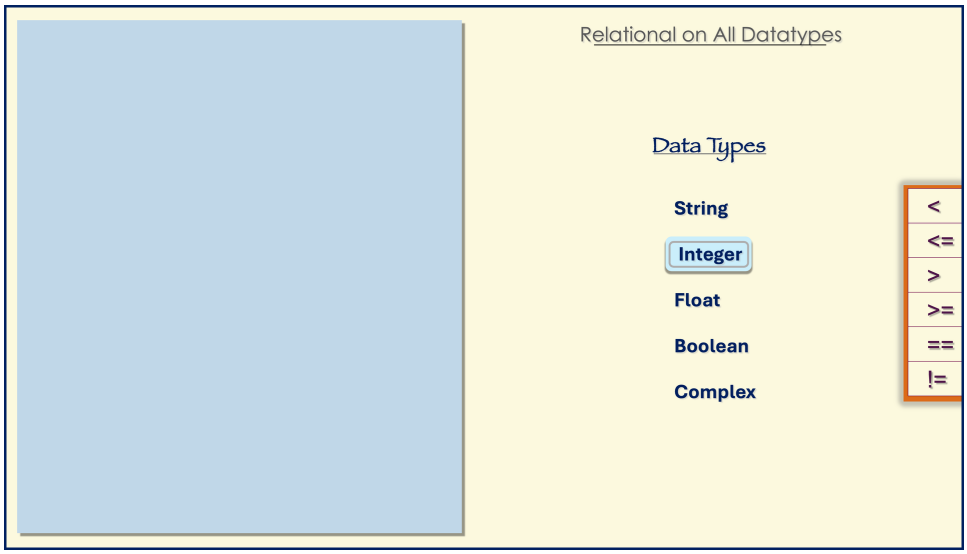

localhost
jupyter PythonPractice
Logout
Trusted
Python 3 (ipykernel)
File Edit View Insert Cell Kernel Widgets Help
Run
Code
In [1]: s1 = 'software'
s2 = 'Hardware'
s1 > s2
Out[1]: True
In [2]: s1 = 'python'
s2 = 'pycharm'
s1 < s2
Out[2]: False
In [ ]: s1 = 'integer'
s2 = 'integer'
s1 == s2
String Comparison
Dictionary Order
s1 = 'software'
s2 = 'Hardware'
True ⟵ s1 > s2
s1 = 'python'
s2 = 'pycharm'
False ⟵ s1 < s2
s1 = 'integer'
s2 = 'integer'
True ⟵ s1 == s2
s1 = 'printer'
s2 = 'print'
True ⟵ s1 > s2
<
<=
>
>=
==
!=

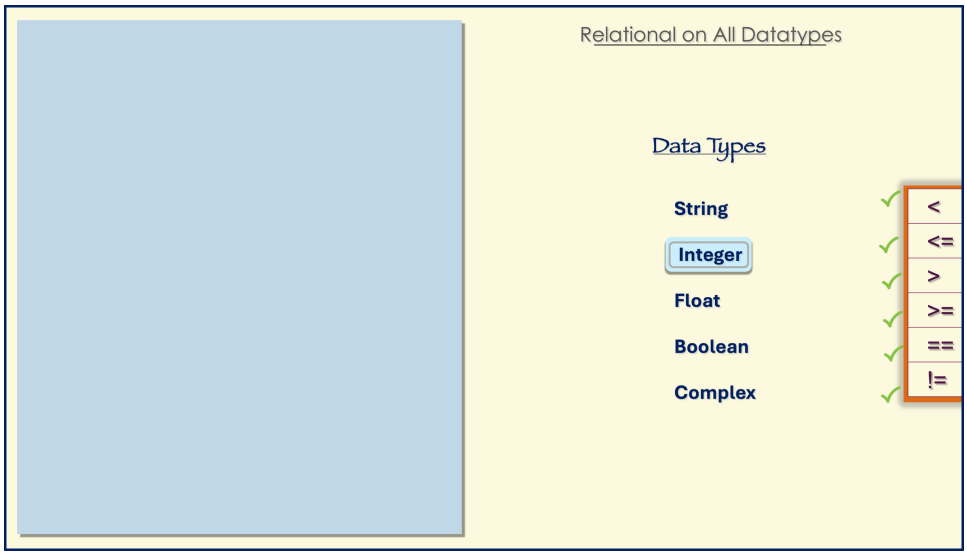

<
<=
>
>=
==
!=

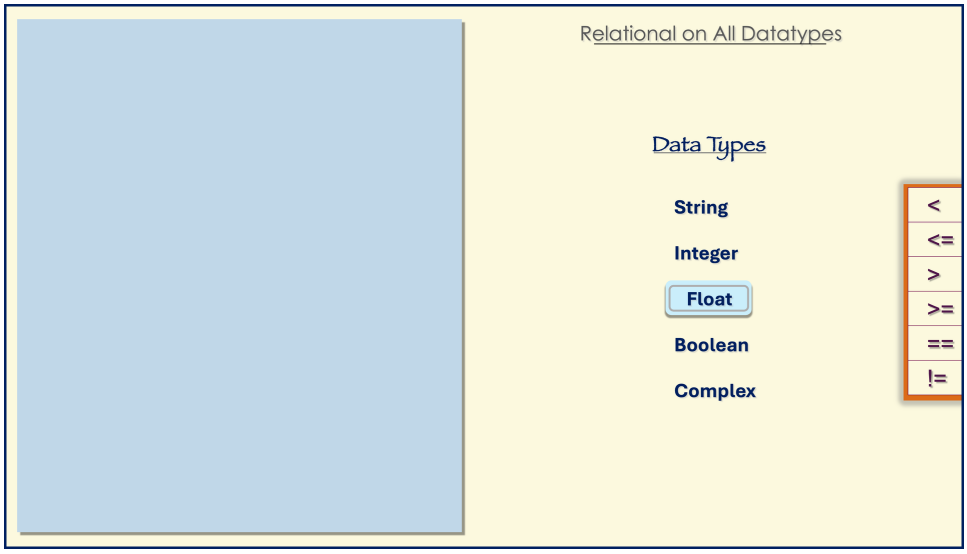

• Relational on All Datatypes
<
<=
>
>=
==
!=

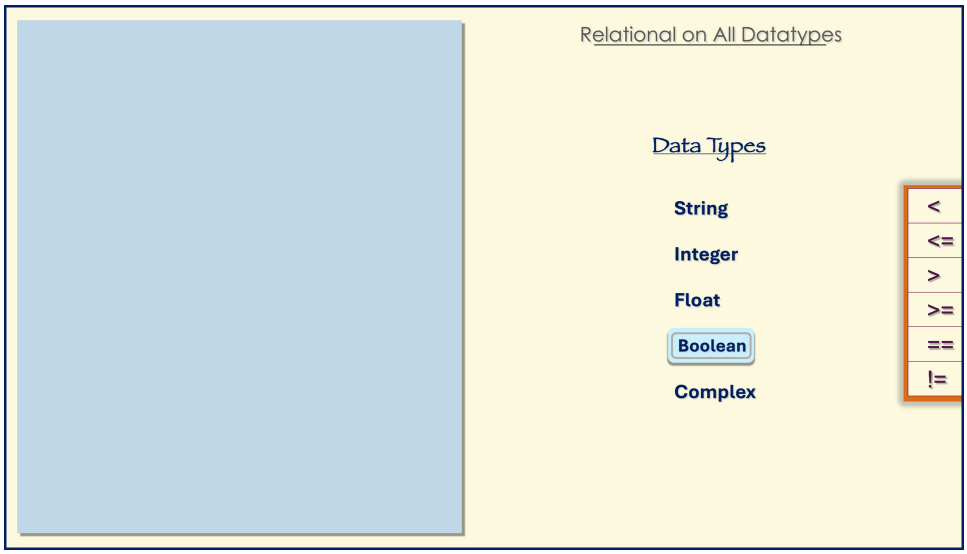

Relational on All Datatypes
<
<=
>
>=
==
!=

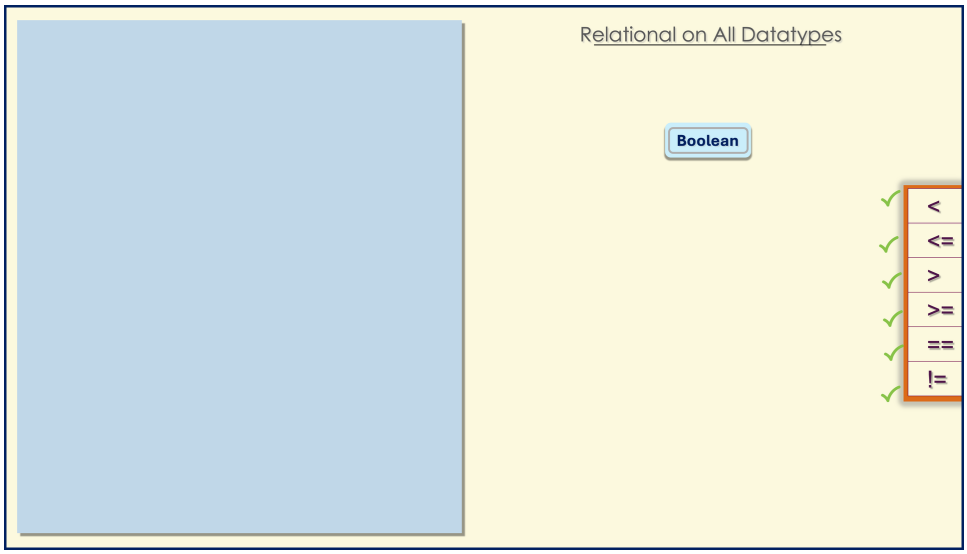

Relational on All Datatypes
Data Types
String
Integer
Float
Boolean
Complex
<
<=
>
>=
==
!=

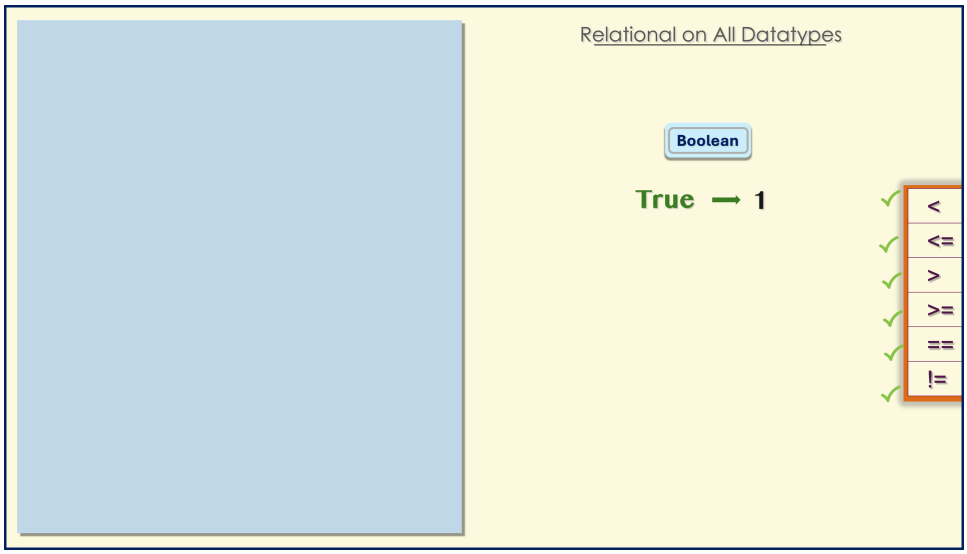

Relational on All Datatypes
Data Types
String
Integer
Float
Boolean
Complex
<
<=
>
>=
==
!=

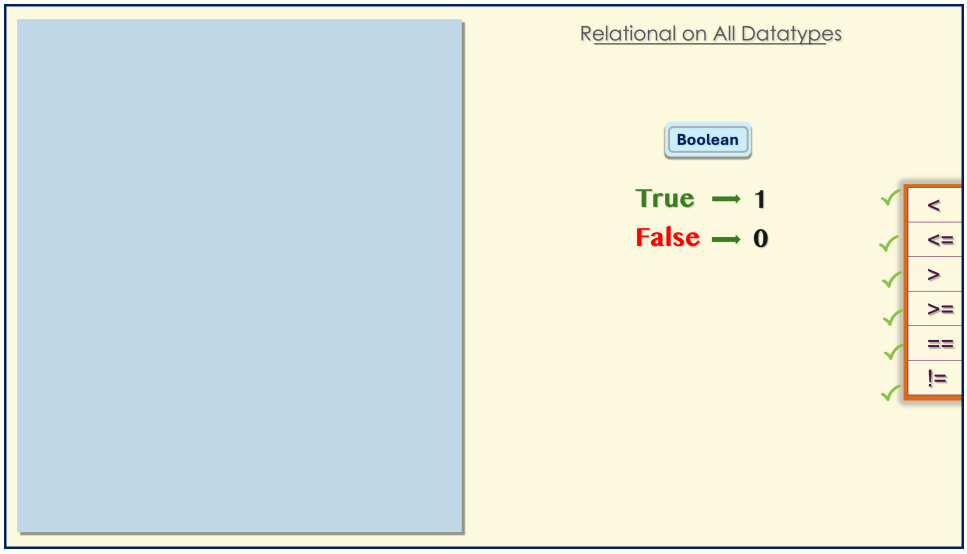

Relational on All Datatypes
Data Types
String
Integer
Float
Boolean
Complex
<
<=
>
>=
==
!=

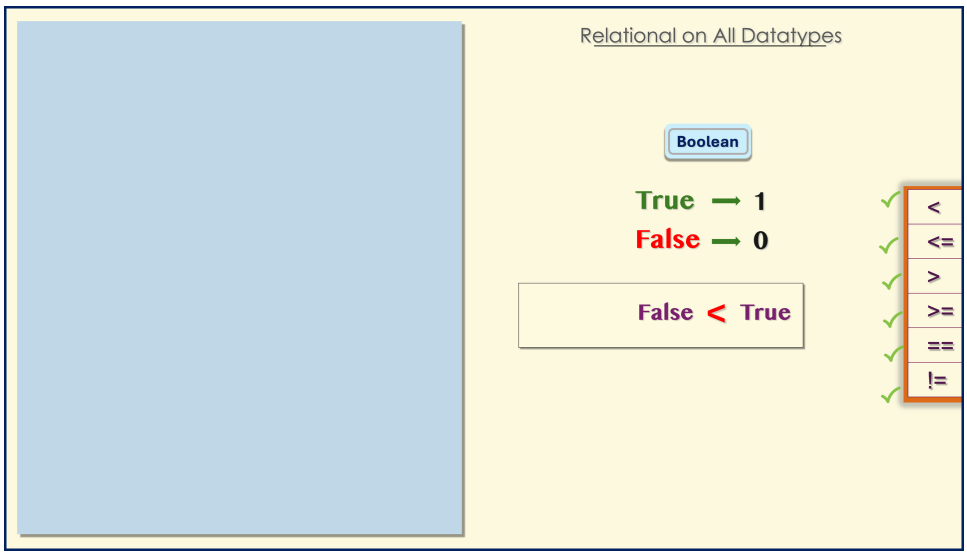

Relational on All Datatypes
Data Types
String
Integer
Float
Boolean
Complex
<
<=
>
>=
==
!=

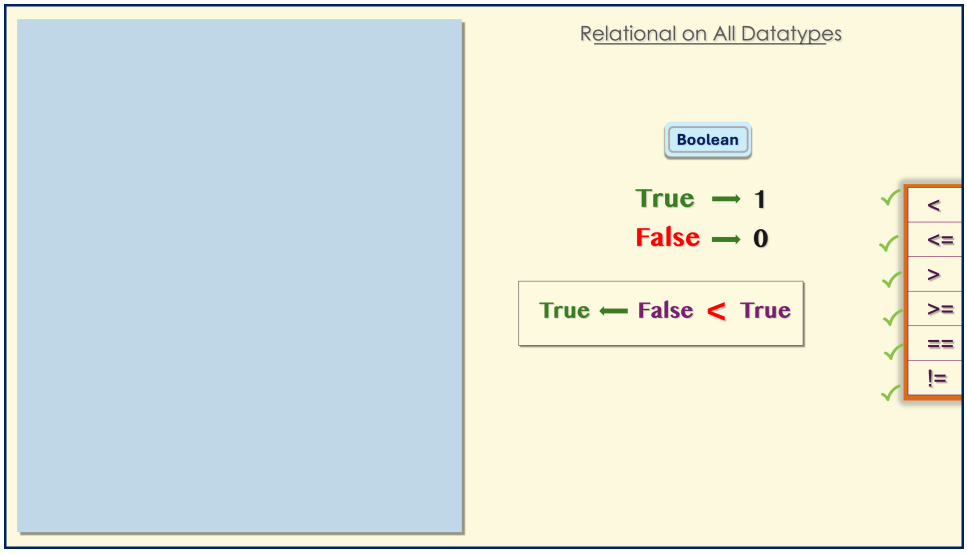

Relational on All Datatypes
Data Types
String
Integer
Float
Boolean
Complex
<
<=
>
>=
==
!=

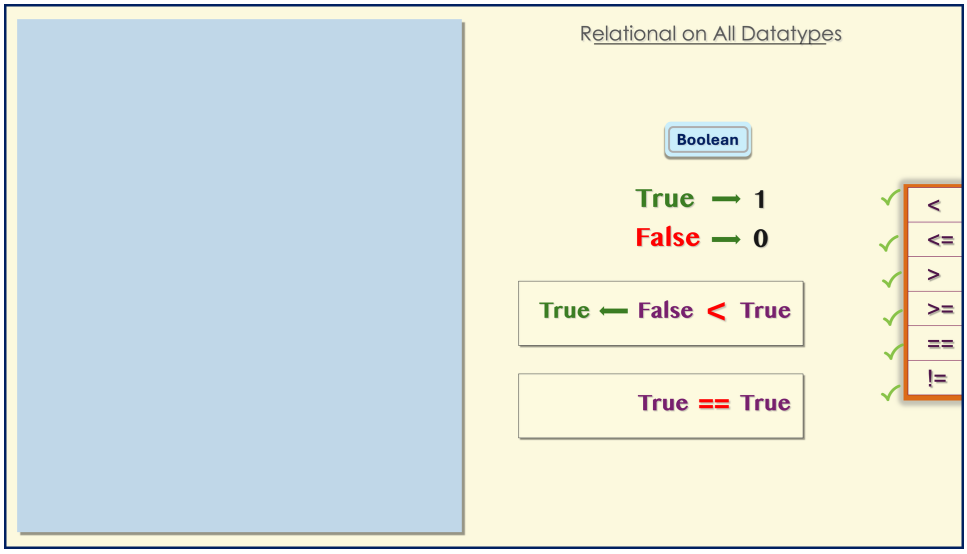

Relational on All Datatypes
Boolean
<
<=
>
>=
==
!=

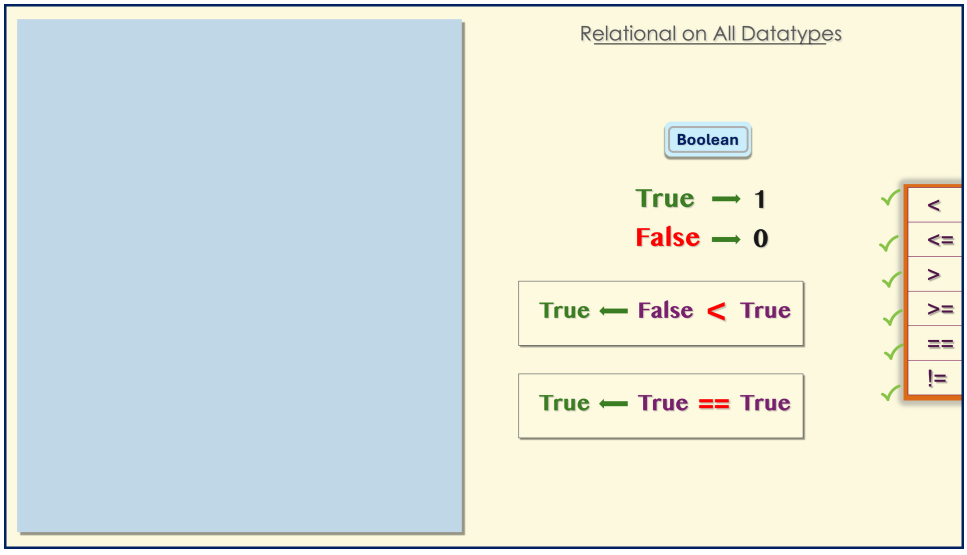

Relational on All Datatypes
Boolean
True → 1
<
<=
>
>=
==
!=

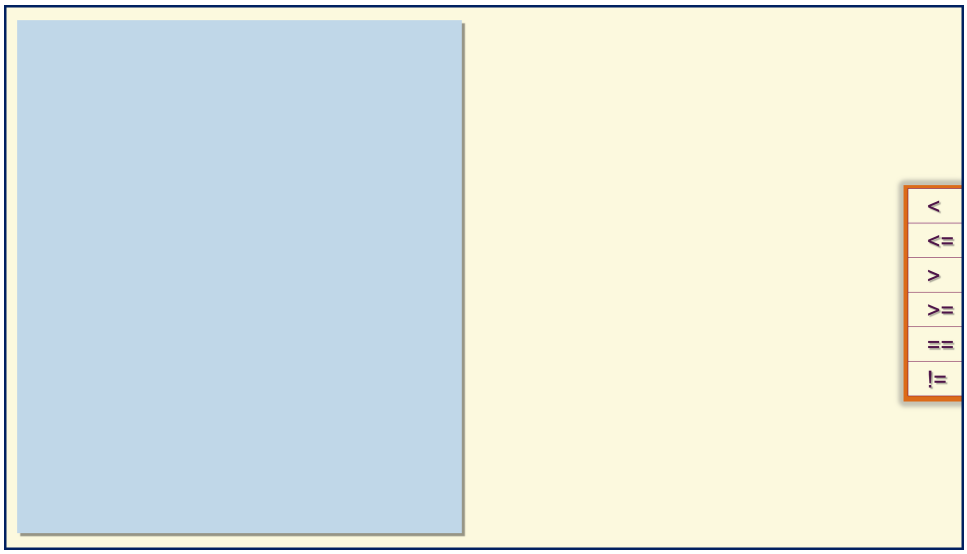

Relational on All Datatypes
Boolean
True → 1
False → 0
<
<=
>
>=
==
!=

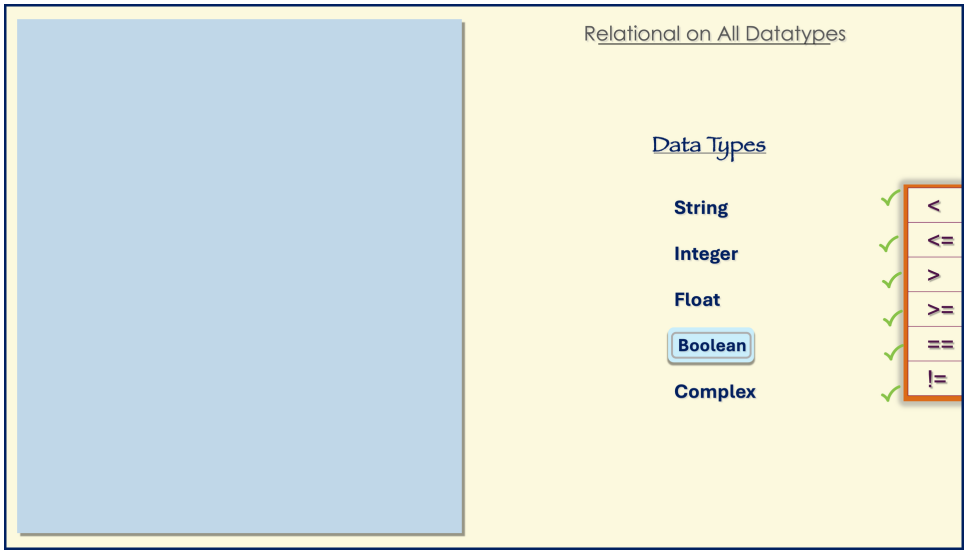

Relational on All Datatypes
Boolean
True → 1
False → 0
False < True
<
<=
>
>=
==
!=

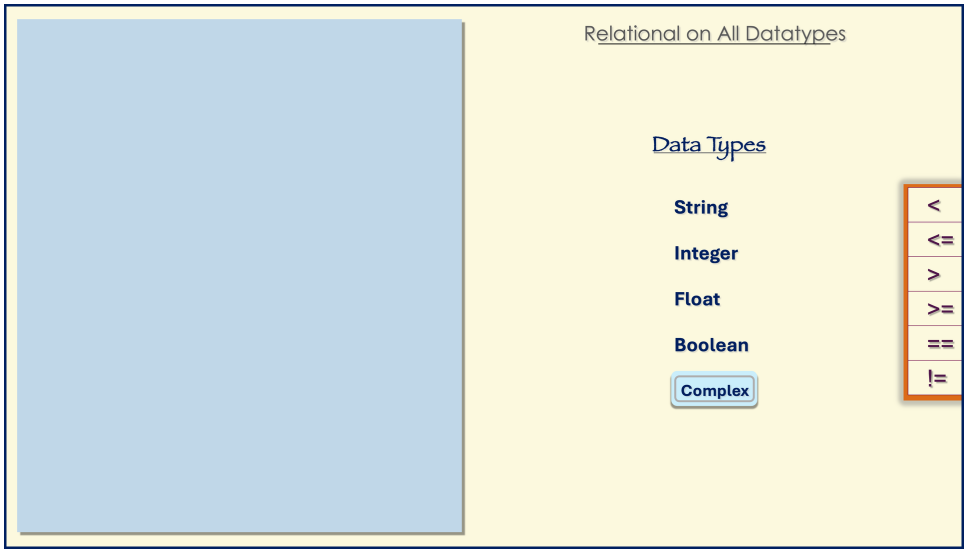

Relational on All Datatypes
Boolean
True → 1
False → 0
True ← False < True
<
<=
>
>=
==
!=

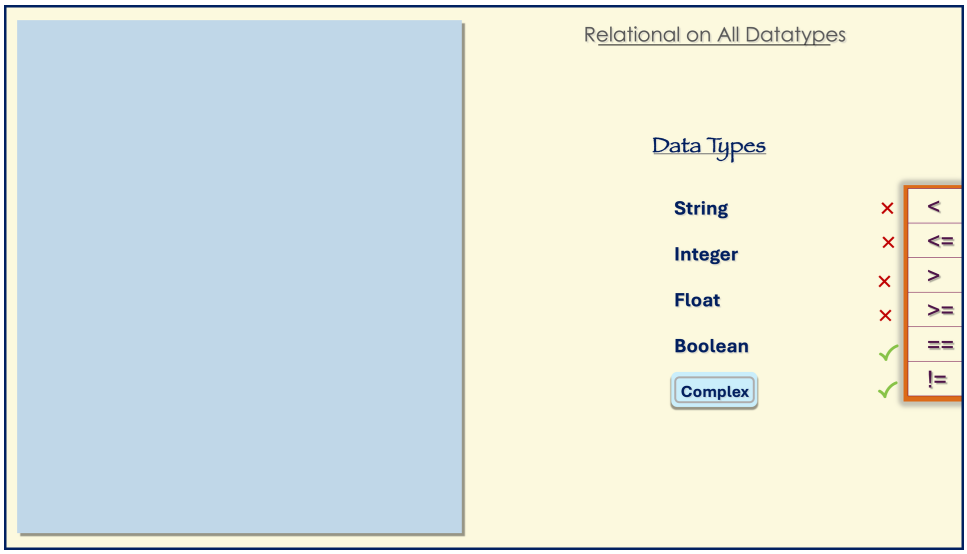

Relational on All Datatypes
Boolean
True → 1
False → 0
True ← False < True
True == True
<
<=
>
>=
==
!=

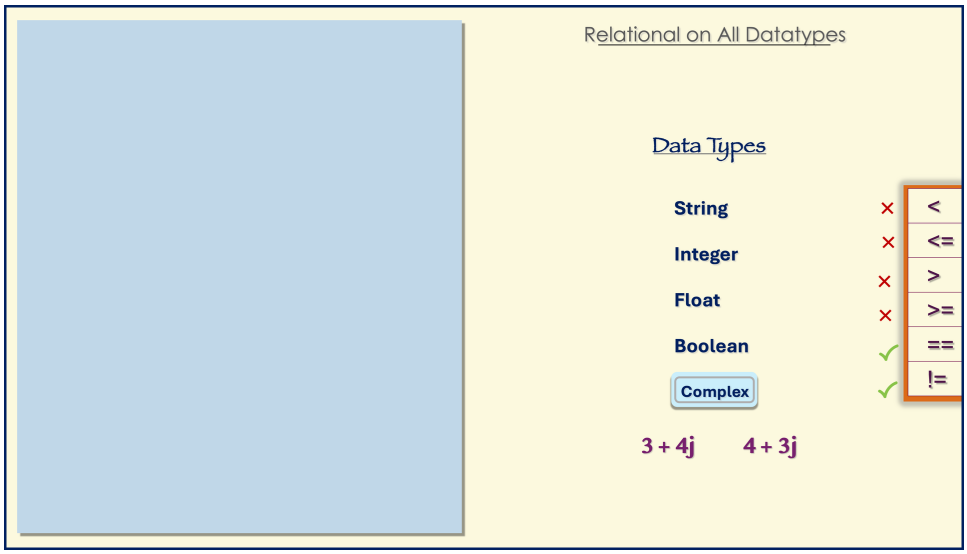

Relational on All Datatypes
Boolean
True → 1
False → 0
True ← False < True
True ← True == True
<
<=
>
>=
==
!=


<
<=
>
>=
==
!=


Relational on All Datatypes
Data Types
String
Integer
Float
Boolean
Complex
<
<=
>
>=
==
!=


Relational on All Datatypes
Data Types
String
Integer
Float
Boolean
Complex
<
<=
>
>=
==
!=


Relational on All Datatypes
Data Types
String
Integer
Float
Boolean
Complex
<
<=
>
>=
==
!=


Relational on All Datatypes
Data Types
String
Integer
Float
Boolean
Complex
<
<=
>
>=
==
!=
3 + 4j
4 + 3j